Curiosity-Driven Discovery of Interference Patterns in Multicore Architectures

Extending ADTool for Reusable Exploration and Experimental Analysis

Arthur Macdonald
Université de Bordeaux, Master 2
Supervisors: Ludovic Matar, Clément Moulin-Frier

Master's Internship Thesis, 2026







Abstract

Multi-core embedded systems allow several programs to run simultaneously, increasing overall performance, but they can also create timing problems: a program may work correctly on its own and still become slower when another program uses the same hardware resources. In real-time systems, this can be critical. To address this problem, the AIxIA project aims to automatically discover and analyse interference between programs using different exploration strategies. I worked within the FLOWERS team at Inria Bordeaux, where Ludovic Matar explores a curiosity-driven exploration strategy called IMGEP. The initial goal of my internship was to integrate this strategy into ADTool, a Python library designed to explore complex systems. This objective later evolved toward a more general system able to take different kinds of code instructions as parameters. During this work, I also identified several issues in ADTool itself, which led me to improve the robustness, usability, and testing infrastructure of the library. I validated both the IMGEP integration and the library rework through tests and metric comparisons. The results show that the new integration works as expected and that the library is now more robust, offers more features, provides better performance, and is easier for researchers to use.

Keywords: curiosity-driven exploration; IMGEP; automated discovery; multi-core interference; real-time embedded systems; ADTool; research engineering.

Acknowledgements

I would first like to thank Ludovic Matar, Clément Moulin-Frier, and Clément Romac for their supervision, their time, and the many discussions that helped me turn initial ideas into a more robust technical contribution. Their different perspectives on exploration algorithms, research questions, and software engineering were especially valuable throughout the internship.

I am grateful to Pierre-Yves Oudeyer for welcoming me to the FLOWERS team, and to all its members for creating an open and friendly research environment, as well as for the discussions and encouragement they offered during these months.

I am especially grateful to Eric Jenn and Clément Moulin-Frier for making it possible for me to contribute to the AIxIA project. I also thank the IRT Saint Exupéry Toulouse team for always being open to discussion, for clearly explaining their project, and for their curiosity about mine. Their thoughtful and pertinent suggestions were invaluable throughout this work.

Contents

Abstract

Acknowledgements

1 Introduction **1**

1.1 Laboratories and research environment 1

1.2 Project and internship context 3

1.3 Internship objectives and scope 4

2 Background and Related Work **5**

2.1 Multicore interference 5

2.2 Curiosity-driven exploration 9

2.3 ADTool: architecture and inherited state 16

3 Work and Contributions **21**

3.1 Work progression and engineering approach 21

3.2 Integrating memory-interference identification 22

3.3 Generalising ADTool to program-based systems 29

3.4 Reengineering ADTool 32

3.5 Resulting contribution to ADTool 36

4 Discussion **37**

4.1 Contribution to the AIxIA project 37

4.2 Collaboration and team interactions 37

4.3 Critical reflection and limitations 38

4.4 Future work and ADTool outlook 39

5 Professional Development and Reflections **40**

5.1 Scientific activities and professional development 40

5.2 Social responsibility, ethics and sustainable development 41

5.3 Use of coding agents and reflections on AI 41

6 Conclusion **44**

Bibliography **45**

A Space size calculations **47**

A.1 Size of the vector retained by the encoder 47

A.2 Upper bound on the number of encoded behaviour vectors 47

A.3 Number of possible initial program pairs 48

B Visualisation and analysis interface **49**

1. Introduction

Modern embedded systems increasingly rely on multicore processors to run more tasks while respecting strict timing constraints. This is useful in fields such as avionics, automotive systems, and robotics, but it also creates a difficult problem: two programs can run on different cores and still interfere with each other through shared hardware resources. A program can therefore be functionally correct and still take longer to execute because another program is using the same cache or memory system [1], [2]. In a real-time system, producing the correct result too late can be as problematic as producing the wrong result.

My internship took place in this context. I worked on the integration of a curiosity-driven exploration method for memory interference into ADTool, a library developed by the FLOWERS team, under the supervision of Ludovic Matar, a Machine Learning Research and Development Engineer, and Clément Moulin-Frier, a permanent researcher. The objective was not only to connect an existing prototype to a software framework. It was also to make its implementation inside ADTool more adaptable to other systems, particularly to code-based systems, and to make it easier for researchers to understand the method and to interpret exploration results.

1.1 Laboratories and research environment

FLOWERS at Inria Bordeaux

My internship took place from early April to late September 2026 in the FLOWERS (FLOWing Epigenetic Robots and Systems) team at Inria Bordeaux. The FLOWERS AI & CogSci lab is at the crossroads of artificial intelligence and cognitive sciences. Its main objective is to understand how humans and machines can continuously discover and learn new skills of increasing complexity: this is the idea of open-ended learning and development. The team also uses this understanding for human-centred applications, especially educational technologies and assisted scientific discovery [3].

More specifically, FLOWERS studies autotelic and aligned intelligence. Autotelic learning refers to the ability of an individual or an artificial agent to set and pursue its own goals. It is a form of intrinsic motivation that can support the progressive organisation of a learning curriculum and creative exploration. The team also studies how language and social interaction can guide this exploration, how these mechanisms can organise at a collective level, and how they can remain aligned with human cul-

tures, values, and preferences. These questions are approached from complementary perspectives: understanding human learning, building curiosity-driven and data-frugal machines, and applying these ideas to education and scientific discovery [3].

My internship was part of the assisted-scientific-discovery side of this work. It used curiosity-driven exploration as a way to help researchers inspect the different behaviours of a complex simulated system.

The FLOWERS team was a very good environment to discover how research projects evolve in practice. Its members had different backgrounds and research interests, but could still bring useful ideas or questions to a problem. This was particularly relevant for my work, which was at the intersection of curiosity-driven AI, software engineering, and low-level hardware. Understanding the scientific literature, Ludovic Matar's prototype, the IMGEP method, the ADTool library, and the hardware problem was therefore an important part of the internship before extending any implementation.

IRT Saint Exupéry Toulouse

The internship was also connected to IRT Saint Exupéry in Toulouse through the AIxIA project. IRT Saint Exupéry is a technological research institute that works with academic and industrial partners on critical systems, notably in the aeronautical and space domains. Its work on critical embedded systems includes real-time applications on multicore platforms and tools for addressing interference, parallelisation, and temporal or spatial segregation issues [4].

Within AIxIA, the IRT Saint Exupéry team developed a complementary research axis to the one carried out at FLOWERS. Christophe Marabotto explored a large-language-model-based approach to interference analysis. Mariem Turki studied the target hardware itself in order to obtain information about its behaviour and build the datasets needed by the project. These two directions were closely connected: one focused on learning and analysing information from data, while the other focused on understanding and collecting that data from the hardware.

Eric Jenn supervised the AIxIA project and followed both IRT and FLOWERS research axes. He was strongly involved in the discussions around the two methods and their possible contributions to interference analysis. This organisation made the project especially interesting: it did not assume that a single approach would solve the whole problem, but explored complementary ways to assist engineers [2], [5].

We held weekly meetings with the Toulouse team to present the progress made during the week, explain the next steps, and discuss potential blocking issues. These meetings were useful to keep track of the different directions, ask for feedback when needed, and understand how my work inside ADTool fit into the wider AIxIA project.

Some meetings also included hardware professionals, who provided more precise information about the software and hardware behaviour and shared their opinion on the ideas being considered.

1.2 Project and internship context

The AIxIA project

AIxIA stands for *Artificial Intelligence for Interference Analysis*. It is part of the AirStrip program, which studies how AI can support system engineering while also identifying its limits. More specifically, AIxIA focuses on temporal interference in multicore embedded systems, especially in aeronautical and space contexts [5]. The project aims to help identify interference sources, characterize the software features related to them, and later estimate or predict their timing effects.

This problem is difficult because execution time cannot always be studied by looking at one program alone. Programs running in parallel can compete for shared caches, memory controllers, buses, or other resources. The precise conditions under which this happens depend on the code, the timing of the accesses, and the target architecture. Consequently, interference analysis is often based on a mixture of manual reasoning, measurements, and specialised tools [1], [5].

The FLOWERS contribution to AIxIA explores this problem as a complex-system exploration task. Instead of starting from a predefined list of interesting programs, the method generates and modifies low-level programs, runs them in a dual-core simulator, and describes their resulting behaviours. The descriptors can include information such as execution time, cache hits, and cache misses [2], [6]. To do so, we use a curiosity-driven algorithm which aims to cover different regions of the behaviour space and discover as many types of interference pattern as possible. The objective is not yet to certify a real processor program timing, but to discover diverse interference behaviours that can later be analysed by hardware specialists. In this context, the way results are represented, clustered, and interpreted is a central question. The exploration workflow is therefore not only about generating and running programs, but also about visualising and pre-interpreting the results.

ADTool: purpose and use in the project

ADTool is an open-source Python library developed by the FLOWERS team for assisted and automated discovery in complex systems [7]. It provides a reusable exploration loop: propose parameters, run a system, describe the observed behaviour, store the result, set a goal, and use the accumulated history to choose the next parameters.

It also provides an interface to inspect and analyse the results of an exploration.

In the AIxIA project, the main goals of using ADTool were to integrate the interference prototype into a reusable exploration workflow, and to make the results of this workflow easier to interact with, reuse, compare and interpret. The prototype was already able to generate programs, run them in a simulator, and describe their behaviour. However, it was implemented as a one-off script, which made it difficult to reuse, adapt, or compare with other experiments. ADTool provides a framework to express the exploration loop in a more structured way, and to add analysis and visualisation tools to interpret the results. This library also acts as a starting point for future work on complex-system exploration, helping researchers more easily develop efficient exploration workflows.

1.3 Internship objectives and scope

The central objective of the internship was to implement the memory-interference study system in ADTool. This goal required an understanding of the main causes of multicore interference, the role of Intrinsically Motivated Goal Exploration Processes (IMGEP) and general curiosity-driven AI, and how the existing prototype represented programs, behaviours, and exploration processes. I then had to adapt and integrate the whole execution loop into ADTool's workflow, while providing metrics to compare the resulting explorations [2].

Early in the internship, Clément Moulin-Frier suggested carrying out this work through a more general *code-based system*. Rather than making the memory-interference case a unique implementation, this abstract system would accept code as parameters, execute it through a selected simulator, and explore the resulting behaviour space. The memory-interference study thus became its first case study.

While implementing this system, I found the code quite difficult to navigate, with several ideas from the original concept either missing, broken, or too closely tied to specific examples. Improving broader parts of the library therefore became another objective of the internship. This work included repairing package imports, removing obsolete or misplaced code, improving the web interface, adding analysis metrics and smoke tests.

This report describes the integration of the memory-interference study into ADTool and the broader changes it required in the library. The memory-interference system is used as a concrete example to demonstrate how ADTool can be adapted to code-based systems, organised into reusable components and used to make inspection easier. It also presents the analysis methods and metrics that were added to compare explorations and help researchers interpret the behaviours produced by the system.

2. Background and Related Work

The work presented in this report lies at the intersection of three areas: timing analysis for multicore embedded systems, curiosity-driven artificial intelligence, and research software engineering. This chapter introduces the concepts needed to understand the problem addressed during the internship.

2.1 Multicore interference

Difficulty of multicore timing prediction

A processor executes the instructions of a program. A *core* is the part of the processor that performs this work: it fetches instructions, carries out computations, and requests the data needed by those instructions. A single-core processor executes one stream of instructions at a time. A multicore processor places several such cores on the same chip, allowing independent programs or tasks to progress concurrently. This makes it possible to host more functions on one computer and to increase overall performance. It is therefore attractive for embedded systems, including avionics and automotive systems [1], [8].

But this gain comes with a difficulty. In an ordinary application, a slowdown may be merely inconvenient. However, in a real-time system, a task must not only compute the correct result but also produce it before a deadline. For example, late sensor processing or a delayed control command could make a system exceed the safety margins of an aircraft or robot.

Ideally, we would like to have a property called *temporal isolation*, meaning that the timing behaviour of one application should not be affected by the activity of another. In practice, however, achieving temporal isolation on multicore processors is very difficult due to applications sharing hardware resources such as caches, memory buses, and main memory [1], [8].

A simplified view of the memory system

Programs constantly read instructions and data from memory and write results back to memory. Directly accessing the large main memory for every operation would be slow. Processors therefore use a *memory hierarchy*: small, fast memories are placed close to each core, while larger but slower memories are placed further away. A cache keeps

recently used data so that a later access can often be served quickly. If the required data is already present, this is a *cache hit* and the request is satisfied. Otherwise, it is a *cache miss*, and the request must be forwarded to the next level of the hierarchy, possibly as far as main memory [1], [9].

Figure 2.1 gives the simplified model used throughout this report. Each core commonly has a private first-level cache (L1), which can serve its own frequent accesses without immediately competing with the other core. At a lower level, cores may share a larger cache (for example L2 or L3), an interconnect that carries requests, a memory controller that schedules access to main memory, and the main memory itself. The simulator used in the interference case study follows this principle with two cores, private L1 caches, a shared L2 cache, and shared DDR memory managed by a controller [9].

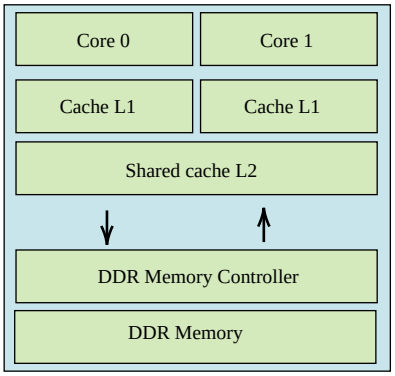


Figure 2.1: Simplified memory hierarchy of a multicore processor used by the simulator. Reproduced from [9].

Sharing these lower levels is a design trade-off. It avoids duplicating large and costly components for every core, while allowing the platform to provide more computing capacity in a compact system. It also means that several cores can access a common main-memory space. The same choice that improves average cost and performance creates an implicit coupling between programs: even when they exchange no data and use different cores, their requests can meet inside a shared component [1].

Interference

In this report, an interference occurs when running a program together with another program changes the former's timing compared with an isolated execution. Let $T(P, \varnothing)$ be the execution time of a program P in isolation and $T(P, Q)$ its execution time while another program Q runs concurrently. The observed timing impact can be written as

$$\Delta T_P(Q) = T(P, Q) - T(P, \varnothing). \tag{2.1}$$

A positive $\Delta T_P(Q)$ is a slowdown caused by concurrent activity. Both programs may still compute exactly the same results; the difference is that one of them completes later. This isolation-versus-co-execution comparison is also the definition used by the curiosity-driven interference study [9].

The shared component where simultaneous use can cause such an effect is called an *interference source* [10]. A configuration in which requests could compete is an *interference path*; it becomes an *interference channel* once the interference has actually been observed. This vocabulary is useful because the mere presence of a shared cache or memory controller does not prove that every pair of programs will interfere: the accesses, their addresses, and their timing all matter [8].

Two complementary effects help explain the phenomenon. *Spatial interference* makes a resource or its contents unavailable to one program, such as when data useful to one core is evicted from a shared cache by another. *Temporal interference* increases the waiting time to use a resource, such as when a request must wait for access to a shared bus or memory bank. In practice, one event can trigger both effects: an eviction creates a cache miss, and servicing that miss creates additional traffic and waiting at lower levels of the hierarchy [1], [8].

Real-time systems and the need for interference identification

In an ordinary interactive application, a slowdown may only be inconvenient. In a critical real-time system, a task must produce the correct result before its deadline. A delay can therefore invalidate the timing argument used to show that the system is safe. The worst-case execution time (WCET) of a task cannot be determined from its execution in isolation alone when its delay also depends on the access patterns of concurrently running tasks [1].

This delay does not necessarily remain local. For example, one core can evict data from a shared cache, causing a cache miss for another core. The miss then creates traffic at lower memory levels and can delay further requests to a shared controller. This practical domino effect means that estimating and measuring the timing of each program alone does not indicate the WCET of their concurrent execution [1].

The relevant behaviour is thus created by the interaction of programs, requests, caches, and scheduling policies. It cannot always be understood by studying each element separately. This is the kind of situation that will be defined later as a *complex system*, and it is why an exploration approach is relevant for this internship.

Multicore processors are not unsuitable for critical systems, but their interference channels and effects must be identified, assessed, and mitigated when necessary. In avionics, AMC 20-193 requires applicants to identify interference channels that could affect software on a multicore processor and to verify the chosen mitigation means.

These channels include shared memory, caches, interconnects, and other shared resources [8].

Main causes of contention

The most immediate cause is *shared-cache contention*. A shared cache has limited capacity and is divided into cache lines. When one core brings data into the cache, it can replace data that another core would otherwise have reused. The affected core then experiences an additional miss and must retrieve the data from the next level of memory. Cache interference is therefore not only about two simultaneous requests: it can also result from an earlier request that changed the cache contents, as illustrated in Figure 2.2(a) [1], [9].

Main-memory contention arises when multiple requests reach shared DRAM or its controller. The controller has finite bandwidth and must arbitrate among requests. Requests that need the same memory bank can be serialized, and extra delays can occur when the controller has to switch the row currently open in a bank. Its scheduling policy can also favour some request patterns over others. The timing impact therefore depends on the detailed sequence of accesses, not only on the number of memory operations, as shown in Figure 2.2(b) [1].

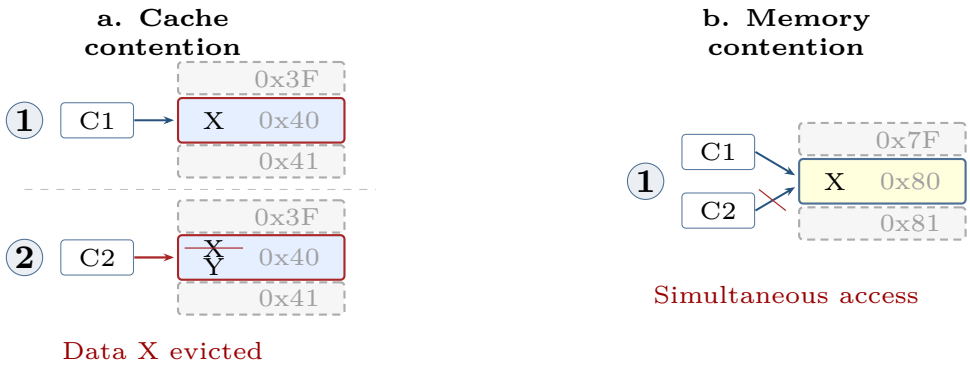


Figure 2.2: Examples of (a) cache contention and (b) main-memory contention.

Other shared elements can also be interference sources. Cores can compete for an interconnect or bus transporting requests, while shared input/output devices can create similar waiting effects. Cache-coherence protocols, prefetching, translation lookaside buffers, and power-management mechanisms can create more indirect effects. Their importance depends on the target architecture, and some details of commercial off-the-shelf processors may not be fully documented [1], [8].

Position of the AIxIA project

Classical identification methods combine manual reading of hardware documentation, measurements using stress or micro-benchmark programs, and model-based analysis. They remain essential, but each has limits: documentation can be incomplete, models require detailed architectural knowledge, and tests cannot easily cover every combination of programs, timings, and hardware states.

AIxIA investigates how artificial intelligence can complement this work for temporal interference in multicore platforms. Its wider scope has three connected objectives: identifying interference sources or channels, predicting their effects, and helping engineers avoid or mitigate them. Its identification work package includes AI-assisted document analysis, automated generation of tests that activate and expose interference, and analysis of execution traces to detect abnormal behaviour and trace it to a cause [5], [8].

My internship contributes to the test-generation direction. The objective is not to certify the timing of a physical processor. Instead, the case study uses a controlled dual-core simulator to generate pairs of low-level memory-access programs and observe their execution descriptors. Curiosity-driven exploration then tries to discover a diverse set of analysable interference behaviours within a limited experimental budget [2], [9]. This controlled case study therefore can both validate the extensions I made to ADTool and provide a reference for complex systems whose parameter space is made of code.

2.2 Curiosity-driven exploration

From curiosity to information seeking

In everyday language, curiosity is the desire to know or discover something that is not yet known. It can be triggered by novelty, uncertainty, surprise, or by the inner feeling that an observation could make a situation easier to understand and anticipate. With this definition, a curious person could explore something even when it does not provide an immediate external reward. In other words, for someone or something considered curious, the information gained from exploration has value in itself, rather than only the final goal or reward.

This distinction is also visible in studies of human and animal behaviour. Novel or surprising stimuli spontaneously attract their attention, and may make them actively choose to obtain information even when this choice does not change their external reward [11]. This does not mean that human curiosity has one unique cause. It is a complex behavioural phenomenon involving attention, learning, motivation, and the

cost of exploring. Computationally, this idea is simplified: an agent is called curious if it selects an action mainly because that action is expected to improve what it knows about its environment.

This idea gives a useful operational definition:

$$\text{curiosity} \approx \text{a tendency to seek experiences with epistemic value,} \tag{2.2}$$

where *epistemic value* means the value of acquiring knowledge. When an artificial system is called "curiosity-driven," it means that its decision process gives value to discovering, learning, or covering unfamiliar outcomes. It does not imply that the system has the subjective feeling of curiosity that a human can report.

Intrinsic motivation and computational curiosity

In artificial intelligence, this idea is usually implemented through *intrinsic motivation*. An external, or extrinsic, reward is supplied by the task or environment, whereas an intrinsic reward is generated from the agent's own observations or learning process.

Consider an information-seeking experiment with two possible actions. The first action reveals a new visual stimulus, while the second does not reveal any new information. Food or monetary rewards can still be given to keep the participant engaged, but they are assigned randomly or as one-off rewards, independently of which action was selected. Both actions therefore have the same expected extrinsic reward. Yet humans and animals consistently choose the action that reveals the unknown stimulus [11]. In this situation, obtaining information is the intrinsic reward: it explains a preference that cannot be explained by the task reward alone.

The same distinction can be implemented directly in a program. In traditional reinforcement learning, the policy, which decides which action to take, is learned from the expected external reward r_t^{ext} depending on the state reached following an action chosen by the agent. A curiosity-driven agent uses the same principle but also computes an intrinsic reward r_t^{int} from its observations, for example from novelty, prediction error, or information gain. The policy is then calculated from both signals: an action can be selected because it is expected to help the task, because it is expected to bring useful information, or for both reasons [11].

In a reinforcement-learning notation, both can be combined as

$$r_t = r_t^{\text{ext}} + \beta r_t^{\text{int}} \tag{2.3}$$

where β controls the importance given to the intrinsic term. This term can reward several different things: visiting a rare state, observing a large prediction error, reducing uncertainty, obtaining information, or making progress at learning a skill [11],

[12].

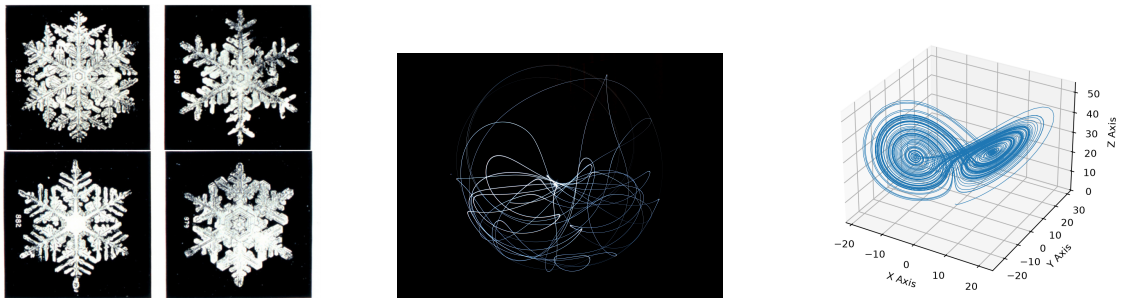
The approach used during this internship does not calculate an intrinsic reward or information gain directly. It uses a different, but very related form of intrinsic motivation. Our objective is not to find the single largest interference delay but to build a broad catalogue of different interference patterns that can later be inspected by hardware specialists. To do so, the exploration algorithm generates target behaviours for itself and tries to reach them. Each target represents a different region of the behaviour space to investigate, rather than an externally specified interference metric to optimise. By repeatedly selecting and pursuing these self-generated goals, the agent is encouraged to discover diverse outcomes. This is an autotelic form of intrinsic motivation: the value of an experiment comes from its contribution to the discovery of the system's behavioural repertoire, not from an external reward for producing one particular result [9], [12].

Exploring complex systems through behaviours

In this report, a system is treated as complex when the relation between an input and its observable outcome is nonlinear and difficult to predict. Complexity can appear even in systems which look simple at first (Figure 2.3). A snowflake is formed by many local interactions between water molecules, yet produces a rich global structure. A double pendulum follows only classical mechanical laws, but small changes in its initial position can quickly lead to a very different trajectory. The Lorenz attractor is another classical example: this simplified model of atmospheric convection is strongly sensitive to its initial conditions, a property that later became known as the *butterfly effect* [13].

Small changes in inputs can therefore create very different outcomes and, at the same time, a large part of the parameter space can lead to similar behaviours (*attractor effect*). Additionally, a complex system has many parameters and interacting components, making it impossible to study its global behaviour by analysing local interactions alone. As illustrated in Figure 2.4, sampling parameters uniformly does not necessarily provide an informative or diverse view of the system [9], [14].

The interference case study has these characteristics. Its inputs are pairs of programs, while its outputs are the measured execution and memory behaviours of the simulated platform. Small changes in instruction order, addresses, or timing can change how requests meet in shared caches and memory, and can therefore produce a very different interference pattern. Conversely, many distinct program pairs can lead to similar observed outcomes. Searching only for the program that produces the largest slowdown would not provide a global view of the different interference mechanisms and behaviours that are reachable. This is why the present work treats multicore



(a) Snowflake (b) Double pendulum (c) Lorenz attractor

Figure 2.3: Examples of complex systems.

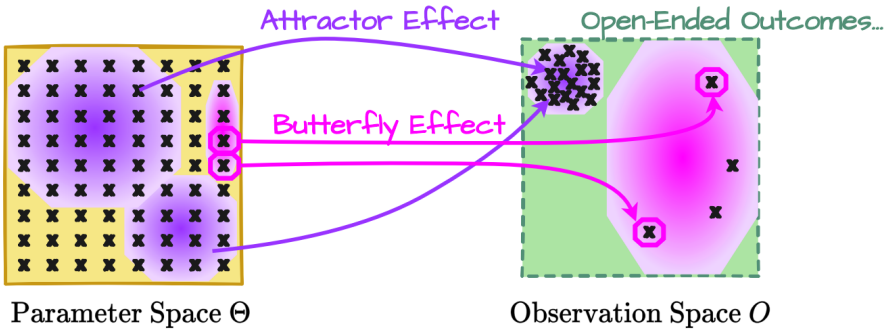


Figure 2.4: Two difficulties of exploring a complex system. The *parameter space* contains the system inputs, while the *observation space* (or the *behaviour space* for simplification) contains the system outputs. Reproduced from [14].

interference as a complex-system exploration problem [9].

The exploration of a complex system can be formalised by separating two spaces. Let Θ be the *parameter space*, B the *behaviour space*, and

$$F : \Theta \longrightarrow B \tag{2.4}$$

be the system mapping obtained by running an experiment. A parameter vector $\theta \in \Theta$ describes an input to the system, while $z = F(\theta) \in B$ is a vector describing the observed behaviour after passing through the system. The behaviour descriptor is deliberately chosen by the scientist: it retains the properties relevant to the scientific question rather than every raw detail of an execution.

For the interference study, θ is a pair of simplified low-level programs executed on the two simulated cores. The corresponding z contains execution and memory-performance descriptors, expressed by comparing concurrent and isolated executions. This makes two program pairs close in B when they have similar measured interference behaviours, even if their instructions differ substantially in Θ [9]. In this context, our agent searches parameters in Θ by manipulating programs, but it evaluates diversity in terms of their consequences on the simulated hardware, in B.

The representation of B is therefore a scientific choice, not a neutral technical detail. It determines what the exploration can distinguish, what counts as a nearby goal, and which patterns will be considered diverse.

Autotelic agents and Intrinsically Motivated Goal Exploration Processes

An *Intrinsically Motivated Goal Exploration Process* (IMGEP) is an algorithmic framework inspired by child development. A child does not receive a complete curriculum of movements, sounds, and object manipulations: it explores, chooses activities to try, and reuses what it has already learned to attempt more difficult ones. IMGEP models this developmental pattern in an artificial agent: instead of trying to optimise a single objective, the agent performs experiments for self-generated goals and retains the resulting data. Data collected while pursuing one goal can then help solve other goals. The strength of this approach is that the agent does not need to know beforehand which goals are reachable, difficult, or useful as stepping stones towards others [12].

An IMGEP agent is called *autotelic* because it selects its own objectives within a goal space, rather than receiving one fixed objective from outside. In the basic IMGEP loop, its goals are target behaviours in B. The agent chooses one target $g \in B$ and then searches for parameters that might produce it. The word "goal" therefore describes a target selected by the exploration process, not a human-like intention [9], [12].

Algorithm 1 presents this basic loop. It contains only the elements needed to organise an IMGEP exploration: a random initialisation, a goal sampler Γ, a parameter generation strategy Π, and the history H of previously observed parameter–behaviour pairs.

Algorithm 1: Simplified Intrinsically Motivated Goal Exploration Process. Adapted from [12].

Require : Parameter space Θ, Behaviour space B, System S
Initialize: Knowledge base $H \leftarrow \emptyset$
while *Initialisation budget not exhausted* **do**
    `// Initialisation loop`
    Sample random parameters $\theta \in \Theta$
    Execute a rollout $\tau \leftarrow S(\theta)$ with $\tau \in B$
    Update knowledge base $H \leftarrow H \cup \{(\theta, \tau)\}$
while *Exploration budget not exhausted* **do**
    `// Exploration loop`
    if *goal refresh* **then**
        Choose goal $g \leftarrow \Gamma$ with $g \in B$
    Retrieve parameters $\theta \leftarrow \Pi(g, H)$ with $\theta \in \Theta$
    Execute a rollout $\tau \leftarrow S(\theta)$ with $\tau \in B$
    Update knowledge base $H \leftarrow H \cup \{(\theta, \tau)\}$
return H

The history H makes the exploration cumulative. It stores every pair of parameters and observed behaviour. During the initialisation loop, random parameters provide the first examples. During exploration, $\Pi(g, H)$ uses the history to retrieve or generate parameters for the current target, and the new observation is immediately added back to H. The interference study also uses this random warm-up to avoid starting from a poor knowledge region [9].

The functions Γ and Π define the particular IMGEP implementation. For interference discovery, the objective is to cover as much of the behaviour space as possible without prior knowledge about its structure, which might have unknown reachable or unreachable zones. Thus, our Γ samples random target behaviours in a surrounding region close to already known behaviours in H. The parameter generation strategy Π then retrieves the k closest known behaviours in space B, takes their associated program parameters, mixes them, and mutates the result to create a new, similar program [9].

Beyond this basic loop, the IMGEP framework can be extended in several ways:

- Γ can sample goals randomly, use an intrinsic signal such as novelty or competence progress, or select goals defined over target trajectories and constrained-

quality objectives;

- Π can be history-based, as in the nearest-neighbour and mutation strategy above, or it can be a learned inverse model or a goal-conditioned policy. Reinforcement learning or neural networks are possible ways to learn such a policy;
- the initial knowledge can come from random experiments or from prior examples, and a human can provide guidance and online feedback during exploration.

These additions do not change the basic process shown in Algorithm 1: it still uses self-generated goals and accumulated experiments to progressively explore a repertoire of behaviours [9], [12], [14].

IMGEP used for complex systems and the interference study

IMGEP was initially developed in the context of developmental robotics. The original question was how an agent could autonomously learn a repertoire of different skills, such as moving objects or producing sounds, without being given a hand-designed curriculum or a ready-to-use dataset. In this setting, the agent selects goals, practices them, and uses the skills already learned as stepping stones for more difficult ones [12].

This idea is also useful for complex systems, but the objective changes. We do not necessarily want to learn a single policy that controls the system. We first want to discover what the system can produce. As explained in the previous subsection, a complex system can have a large parameter space, nonlinear dynamics, attractors, and a strong sensitivity to small changes in its inputs. Randomly sampling parameters can therefore repeatedly produce similar outcomes while missing other regions of the behaviour space. IMGEP instead uses observed behaviours to organise the next experiments, which helps build a behavioural catalogue within a limited experimental budget [9], [14].

For the interference study, the complex system is the simulated dual-core platform. Its parameters are pairs of simplified programs, and its behaviours are vectors built from execution time and memory-performance descriptors. The goal is not to find only the program pair causing the largest slowdown. It is to discover many different interference patterns that may later help hardware specialists identify the involved mechanisms [9].

The interference-study IMGEP uses a multi-parent retrieval strategy. After its random warm-up, it samples target behaviours inside a region defined from the current history and slightly extended towards unknown areas. It then retrieves the closest known behaviours, reuses their corresponding programs, and mixes and mutates them to create a new candidate. The simulator returns a new behaviour descriptor, which is added to the history before the next target is selected [9].

In this context, curiosity does not mean that the agent randomly generates programs forever, nor that it focuses only on the strongest interference. It means that the agent repeatedly creates and pursues unknown target behaviours to extend its knowledge of the system's behavioural repertoire. In the supplied interference study, this method produces broader and more uniform coverage of its measured behaviour space than random program generation for the same experimental budget [9].

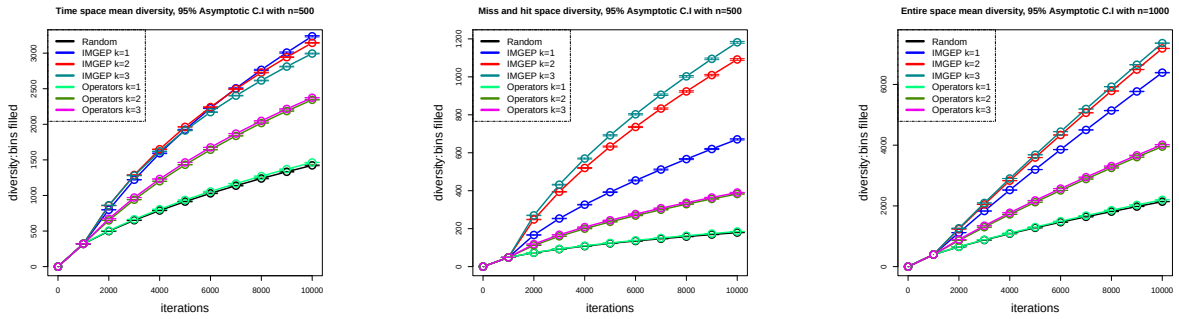


Figure 2.5: Diversity coverage during interference exploration. The black curve is random program generation. The six remaining curves are IMGEP-related configurations with different parameters. Reproduced from [9].

2.3 ADTool: architecture and inherited state

ADTool, short for *Automated Discovery Tool*, was created to make curiosity-driven exploration usable beyond one particular scientific experiment. It is a Python library and a web interface developed in the Inria FLOWERS team for running experiments, saving their discoveries, and inspecting the resulting behaviour space [7], [15]. Its purpose is to support the study of a system for which the parameter space is large and the behaviour space is initially unknown or difficult to predict. Instead of manually trying parameter values one after another, a scientist can use an exploration algorithm to progressively build a behavioural catalogue.

The tool does not decide what is scientifically interesting by itself. A scientist defines the system, its parameter space, and the behaviour descriptors retained from its raw outputs and ADTool then automates the repetitive part of the process: generating parameters, running the system, recording a behaviour, and using the history to generate a new parameter set. The resulting catalogue remains available for human inspection which is particularly useful when simulations are expensive, or when the relation between parameter space and behaviour space is too irregular to explore manually [14], [16].

Intended architecture and exploration loop

It is important to distinguish the architecture ADTool is meant to provide from the state of the inherited implementation. In principle, ADTool separates a domain-specific experiment from the generic exploration procedure. At the centre of an experiment is a *System*. It receives the current parameters, performs one rollout or simulation, and returns a raw result. A system can also render an image or video, because a compact numerical descriptor is useful to an algorithm but is rarely enough for a scientist to understand a discovery [7], [17].

Two maps connect the system to the explorer. The *Parameter Map* defines the parameter space: it samples random parameters for the initialisation and provides the mutation operation used to make a nearby candidate. The *Behaviour Map* transforms the raw system result into the smaller behaviour vector used by the algorithm, and samples target behaviours in the behaviour space. These components correspond directly to the formalisation $S : \Theta \to B$: the parameter map operates in Θ, the system executes S, and the behaviour map constructs the representation used for B [17].

The default explorer is an IMGEP. After random initialisation, it samples a random goal through the behaviour map, compares this goal to the history with Euclidean distance in behaviour space, retrieves the parameters associated with the nearest known behaviour, and applies a mutation to obtain the next parameters. Callbacks can save a discovery, report progress, or react to the end of a run. The intended execution cycle is therefore the general IMGEP loop of Algorithm 1, expressed as reusable software components rather than as code tied to one system [12], [17].

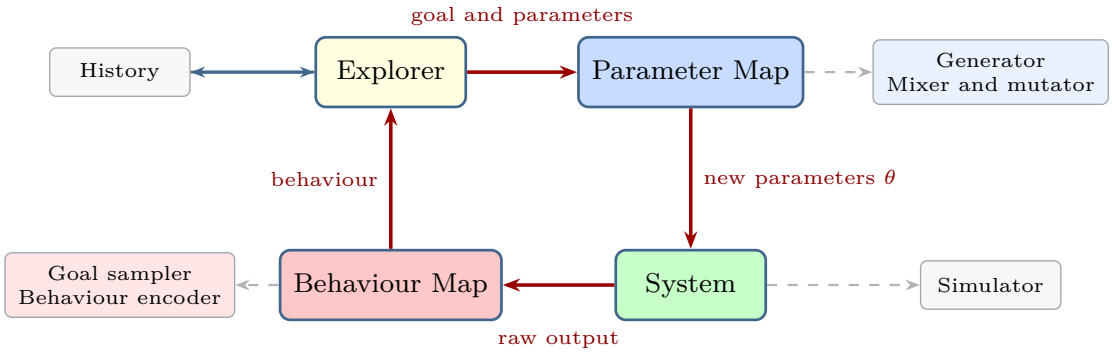


Figure 2.6: ADTool exploration loop. Red arrows show the main loop, blue arrows history access, and dashed gray arrows supporting components.

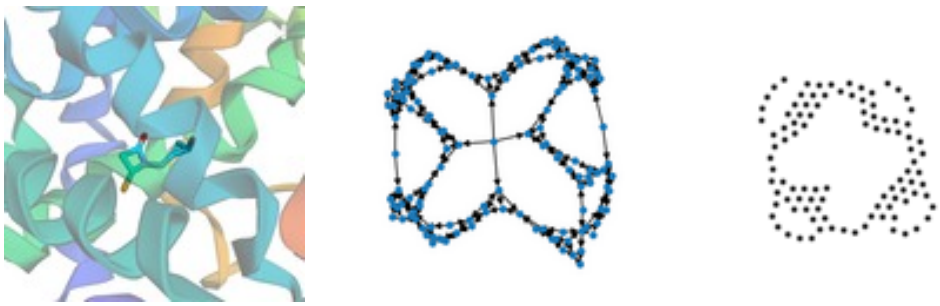
Figure 2.6 shows this intended structure. In this design, a new complex system is supposed to require only a configuration file, a system, a parameter map, and a behaviour map, rather than a modification of the exploration algorithm itself. The

configuration gives the Python paths of the concrete classes to instantiate. It can select another IMGEP implementation with a different goal sampler, nearest-neighbour retrieval, mutator, or additional exploration steps, and can also select a non-IMGEP explorer. This is the central idea behind ADTool: the common experiment loop remains reusable while both the system and the exploration method can be adapted to a scientific question [17].

The intended workflow also includes saving each discovery to disk and displaying it in a local web interface. A researcher can then monitor an exploration and select results for closer analysis [7], [17]. The historical repository already contained several examples based on this idea, presented in the next subsection. Their presence shows the intended range of ADTool, but it does not mean that the inherited implementation fully matched this architecture.

Systems already represented

The historical repository contained several demonstrations of this intended generality. Most were dynamical systems in which visually different patterns emerge from a small set of rules: Gray–Scott reaction–diffusion, Lenia, Particle Lenia, Flow Lenia, Flash Lenia, a reversible block cellular automaton, Kuramoto oscillators, and an N-body simulation. These systems are pertinent examples of systems where parameter spaces can be large while the resulting patterns are difficult to predict from the parameters alone. The repository also included protein-pocket docking, Rectified Kuramoto patterns, hypergraph rewriting in Wolfram Physics, a synthesiser, and variations of a Stable Diffusion inference. Together, these examples show the intended generality of the exploration structure across very different systems. However, their presence did not mean that this flexibility was actually easy to use in the inherited library [17].



(a) Molecular docking (b) Hypergraph rewriting (c) Particle Lenia

Figure 2.7: Three examples already represented in ADTool

Limitations of the inherited implementation

The ideas behind the architecture were useful, but the actual library was still a research prototype. The main problem was not the IMGEP method itself, it was that the library was not as modular as it was supposed to be. Generic code, code made for one particular example, optional services, and unfinished features were all mixed together. For example, the core library contained a CPPN/NEAT implementation, document and media conversion tools, several persistence backends, and wrappers which were not needed for the main exploration loop. At the same time, some example-specific code was not clearly separated from the reusable code. This made it hard to know what was really needed to create a new system, what was only useful for one example, and what was simply old code left in the repository [17].

The configuration system had a similar issue. It was meant to make the library easy to adapt: a user could give the paths of the classes to use in a JSON file. But the classes were loaded dynamically, and some objects were then created inside the explorer. This gives flexibility, but it also means that a small mistake in a path, a missing dependency, or an incompatible constructor can only be found when the experiment is launched. The package itself and the commands used to run the examples were also not clearly separated. Therefore, using ADTool as an external library, as opposed to running it from its own repository, was barely feasible [17]. Even when used from the repository, the library was not straightforward to use.

The history was probably one of the most important issues. As shown in Algorithm 1, the history is the main memory of IMGEP: it stores the old parameter–behaviour pairs which are used to choose the next parameters. The original structure was supposed to store this data in an on-disk database through the `Leaf`/locator system in order to avoid filling the RAM during long and costly experiments, while also saving the states of the different components and allowing checkpoints to be restored later. For this, the database transformed the data into serialised objects before saving them, but these transformations were very slow to compute, and the database itself was not working correctly, breaking the checkpoint feature. In the end, the history was kept in RAM anyway, while still paying the cost of the slow serialisation process and database access failures.

Finally, the library had very little verification for something meant to work with many different systems. The original repository had a small test asset for a file-conversion utility, but no real tests for the provided explorations. A lot of the information needed to use the library was in JSON files, notebooks, or example directories rather than in a stable documented interface. There was also no general set of metrics to evaluate the overall performance of a rollout. The tool could save and display individual discoveries, but it did not provide a common way to measure how the be-

haviour space was covered, how this coverage evolved during an experiment, or how an exploration method compared with a random baseline.

The old web interface (Figure 2.8) was useful to display discoveries, but it was also very limited. It did not provide enough interactivity to navigate an experiment, filter, or easily compare discoveries. This is troublesome for large experiments: being able to see the results was not enough to actually inspect and understand them. It also could not tell if an experiment, a reload, or a configuration still worked as intended.

So, ADTool already had the important idea: separate the system, the parameter map, the behaviour map, the explorer, the history, and the human analysis. This is exactly what makes it relevant for the exploration of multicore interference. But this separation was clearer in the intended architecture than in the actual code.

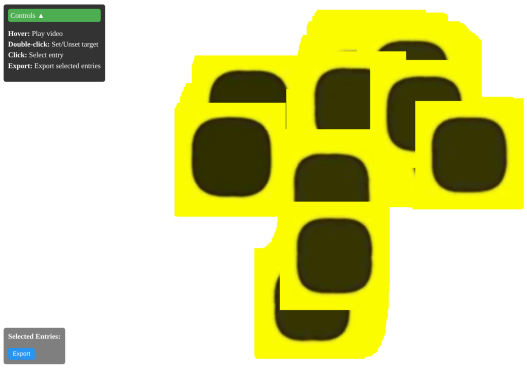


Figure 2.8: Default and only view of the old ADTool web interface with little functionality. In yellow are some discoveries for a Gray–Scott system.

3. Work and Contributions

The initial objective of my internship was to integrate the existing memory-interference prototype into ADTool. However, completing this integration also showed which parts of the library were difficult to reuse in practice. My work consequently evolved from the implementation of one new experiment toward the construction of a more general program-based exploration layer and a wider rework of ADTool itself.

This chapter presents this progression in the same order as it happened. I first describe how I understood and reproduced the reference interference experiment. I then explain how I separated this case study into reusable components for program-based systems. Finally, I present the changes made to the core library, history, checkpoints, analysis tools, visual interface, and verification infrastructure.

3.1 Work progression and engineering approach

The internship began with a learning phase. Although I already had experience with Python and machine learning, this subject also required an understanding of low-level memory architectures, multicore interference, curiosity-driven exploration, and the internal organisation of ADTool. I therefore started by reading the supplied articles and theses, inspecting Ludovic Matar's simulator and exploration prototype, and studying the existing ADTool framework. This phase allowed me to understand every concept required for the work. The knowledge I acquired is the basis of the simplified presentations given in the previous chapter.

The Master also gave me a useful starting point for this learning phase. Its courses introduced me to several families of AI methods and trained me to adapt to unfamiliar concepts, compare possible approaches, and consider their limits. Earlier courses on computer architecture and hardware organisation were particularly helpful to understand how memory hierarchy, caches, and interconnects work, and how they can create interference between programs.

With this foundation, I was able to establish a first plan, and the project naturally progressed through four main phases:

1. I reproduced the reference interference exploration inside ADTool. This required modules for generating and modifying programs, executing the simulator, encoding its outputs, sampling goals, and running IMGEP.

2. I added verification and analysis tools. The first coverage results exposed dif-

ferences between my implementation and the reference prototype, while smoke tests made it possible to change the library without repeatedly checking every example by hand.

3. I generalised the interference-specific components into a program-based-system layer. The interference study then became one configured case of this layer instead of defining the reusable abstractions itself.

4. I reworked the surrounding library. This included package imports, separation between the core and examples, history and checkpoints, the web interface, analysis modules, configuration, documentation, and removal of obsolete code.

These phases were not always strictly separated. For instance, I first modified the visualisation while trying to inspect the interference results, then returned to it after the analysis modules were better defined. The history followed a similar process: I implemented an initial replacement, discovered that it did not preserve all the intended checkpoint semantics, and rebuilt it after discussions with Clément Romac. These kinds of interactions were an important part of the work. The inherited code did not always make it clear whether a behaviour was intentional, unfinished, or no longer used. I consequently tried to first recover the intended purpose, propose a simpler design, discuss it with the relevant researchers, and only then make a larger modification.

The main implementation decisions were my own proposals, but they were not made in isolation. I discussed the interference model and analysis metrics frequently with Ludovic Matar. I discussed the generalisation to program-based systems and tests with Clément Moulin-Frier, and the internal structure of ADTool, such as history or the web interface, with Clément Romac. This working method was necessary and interesting because each of them viewed the same library from a different perspective.

3.2 Integrating memory-interference identification

From a reference script to an ADTool experiment

Ludovic Matar's prototype contains the main scientific loop: generate two simplified programs, execute them in a dual-core simulator, transform the simulator output into a behaviour vector, and use IMGEP to generate the next programs. My first contribution was to express this loop through ADTool's components instead of keeping it as a single use-case script. The implementation therefore had to respect both sides: it had to reproduce the reference method, but also follow ADTool's separation between the system, parameter map, behaviour map, and explorer [6], [7].

In the implemented representation, one candidate contains one program for each

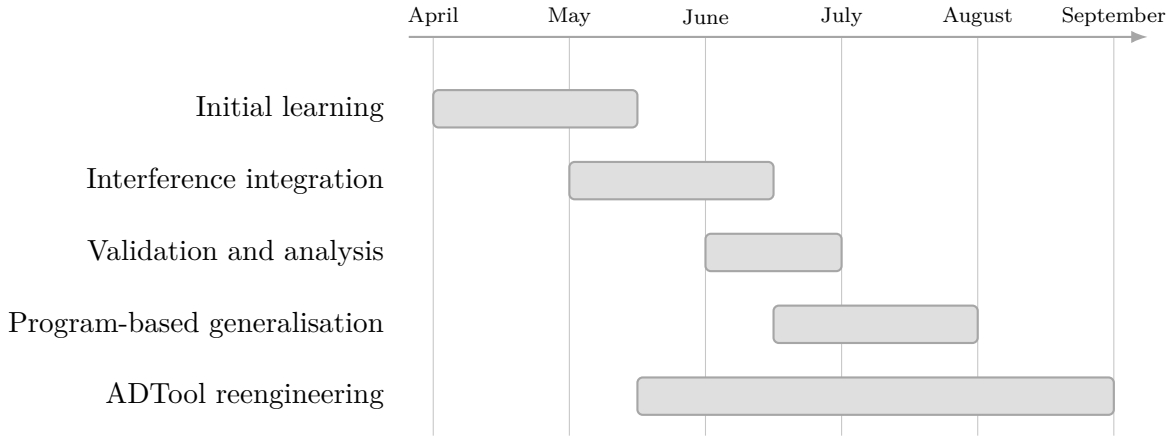


Figure 3.1: Main phases of the internship. Several phases overlapped as problems discovered during integration led to changes in the library.

core. A program is a simple dictionary associating an execution cycle with an instruction composed of an operation and a memory address. The available operations are strictly either *read* or *write*, since the only behaviour we want to model is memory interference. We do not need to represent arithmetic or other operations. A simplified candidate can consequently be represented as:

```
core0 = {3: ("read", 4),  9: ("write", 17), ...}
core1 = {2: ("write", 31), 8: ("read", 25), ...}
```

This representation keeps the information that matters for the simulator: what memory operation is requested, at which address, and at what point in the program, while keeping the representation very simple. It is also structured enough for a generator or mutator to modify one part of the instruction without having to struggle with rewriting or dependency issues. Before a program is sent to the simulator, a normalisation step checks the cycle keys, instruction shape, operation, and address types, to ensure that any candidate retrieved from the history or generated by a new operator is valid.

The system does not execute only the concurrent pair. For each candidate, the simulator runner performs three executions:

1. the program of core 0 is executed alone;
2. the program of core 1 is executed alone;
3. both programs are executed concurrently on the two simulated cores.

The two isolated executions provide the reference needed to interpret the third one. The runner can then compute, for each core, the additional execution time and memory

events observed during co-execution. This directly follows the definition of interference introduced in Section 2.1: the relevant result is not only the raw concurrent execution time, but its difference from the isolated case.

The simulator models private L1 caches, a shared L2 cache, an interconnect, a DDR controller, and banked DDR memory. Its outputs contain execution times, L2 hits and misses, and more detailed read and write events for DDR rows and banks. My integration keeps the simulator behind a runner interface. The ADTool system is therefore responsible for receiving and validating the parameter payload, while the runner is responsible for the execution protocol and the simulator remains responsible for the hardware model. During discussions with Ludovic Matar, we considered that a later version of the experiment could use a more detailed simulator such as *gem5* in order to move closer to a real hardware execution pipeline [18]. This was the main reason why I separated these tasks.

Creating programs

The creation of a new parameter of interest relies on four interchangeable submodules: a generator, a goal sampler, a mixer, and a mutator. Each module is selected in the experiment configuration, which makes it possible to compare exploration strategies without changing the rest of the system.

Generator At bootstrap, the generator is used to create a first knowledge base of programs. The number of instructions, the cycle range, and the address intervals of the two cores are set by the user in the configuration and, within these limits, the generator randomly samples execution cycles, read or write operations, and addresses. The idea is to cover the easier zones in the behaviour space roughly and quickly to avoid falling into behavioural blind spots.

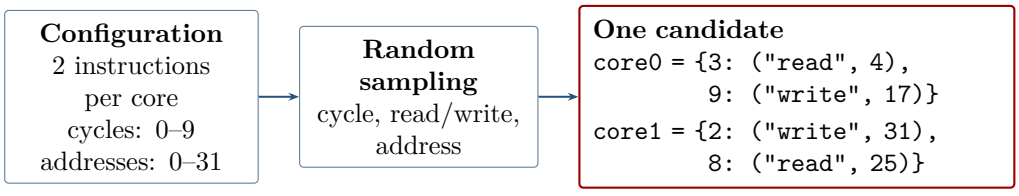


Figure 3.2: Generator: sample a program pair within configured limits. Each dictionary maps execution cycles to (operation, address) pairs.

Goal sampler Once the first results have been stored, the goal sampler is used to propose new behavioural targets for the explorer. By continuously selecting targets in different areas and directions of the behaviour space, one guides the search towards a

progressively broader coverage of this space. IMGEP then compares the selected goal to the behaviour history using normalised distance, finds the closest known behaviours, and retrieves the programs associated with them.

The initial sampler, provided by Ludovic Matar's reference implementation, draws random goals from the bounds observed in the archive, with a small expansion beyond these bounds. I also implemented and tested a Quasi-Monte Carlo sampler based on a Sobol sequence [19]. It uses the same bounds, but places successive goals more evenly than independent random draws. The sampler and the frequency at which new goals are selected are chosen in the configuration.

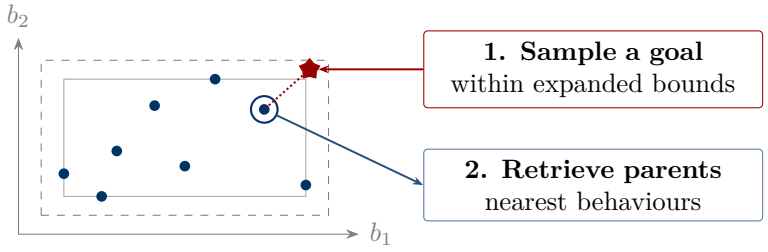


Figure 3.3: Goal sampling, followed by IMGEP parent retrieval. This schematic normalised behaviour space shows one nearest neighbour. Known behaviours are represented by blue dots, the selected goal by a red star.

Unlike the parameter spaces of other complex systems, that can be described by numerical vectors, the programs used here do not support simple arithmetic operations. Averaging read and write instructions, or two discrete memory addresses, does not really make sense. We therefore needed custom mixers and mutators adapted to this program-based parameter space.

Mixer The number of retrieved neighbours and the mixing strategy are chosen by the user in the configuration. If several parents are requested, a mixer first builds one parent program from them. Several strategies were implemented for this purpose such as the *chunk mixer* which divides time-ordered programs into consecutive parts and chooses a parent for each part or the *preserve time structure* strategy which keeps the relative timing inside each selected part as far as possible when combining them. The idea behind this mixing strategy is to try to create a new child parameter with inherited behaviours from the selected parents, similarly to a genetic operator.

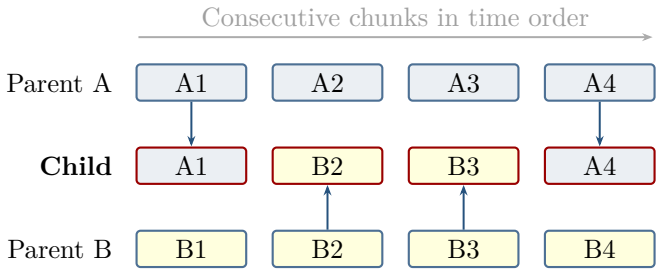


Figure 3.4: Example of a mixer: *Chunk Mixer*. It assembles a child from consecutive parts of two parents. Each block represents a chunk of instructions.

Mutator The newly selected or mixed parameter is finally mutated to form the next candidate. The user configuration sets how many mutation operations are applied and, at each operation, the mutator randomly chooses to add an instruction, delete one, or modify one by changing its access type, its address, or both. New cycles and addresses are drawn in the same configured ranges as during generation and some additional constraints are also applied to ensure that the resulting program is still valid. The key here is to have a kind of evolutionary mutation operator which allows the mutated program to remain close to a program whose behaviour is already relevant to the selected goal, without having to force a strict trajectory, letting the algorithm explore the behaviour space in a flexible way.

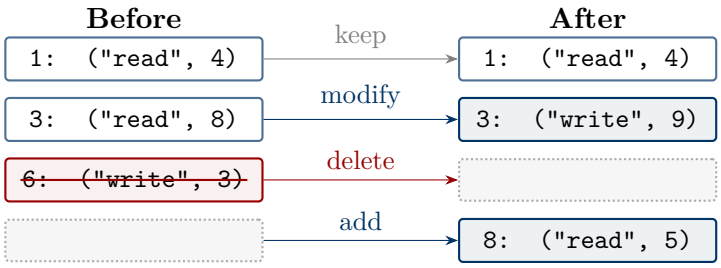


Figure 3.5: Example of mutator operations: three edits to one core's program. Dotted boxes indicate the absence of an instruction before an addition or after a deletion.

Keeping these modules well isolated made their effects easier to compare and evaluate. It also avoided placing interference-specific decisions inside the generic IMGEP explorer: replacing a component does not require modifying the system, behaviour encoder, or exploration loop, but simply changing a path in the configuration.

Encoding interference behaviours

The raw output of one simulation contains more information than is useful for guiding the exploration. Keeping every raw counter would give importance to dimensions un-

related to interference and would make the behaviour space harder to explore. Therefore, we have an *interference behaviour encoder* which retains only the information estimated to be relevant to the study, and flattens it into one numerical vector. The selection is configurable and the default encoding includes:

- amount of L2 read and write hits and misses for each core, computed by comparing mutual execution with isolated execution;
- DDR row-buffer hits and misses for each core, also computed by comparing mutual and isolated executions, retaining the row and bank of each event;
- the completion time of each core individually during isolated and mutual execution, and the completion-time difference between the two cores in that mutual execution.

These features represent the difference between the statistics collected during mutual execution and those collected when each core runs in isolation. They are therefore descriptive features for exploration rather than a direct attribution of individual interference events to either core. For L2, the representation keeps aggregate read/write hit and miss counts. For DDR, it additionally preserves the row and bank associated with each count, because this information is relevant to row-buffer conflicts while remaining manageable in the simulated configurations. The three timing values compare the two cores' overall execution times individually and together, giving a global view of the interference effect.

Overall, these values produce a vector of a fixed size for one given configuration. During the internship, it generally contained between 50 and 100 dimensions because the simulator configurations remained small. In more realistic configurations, the number of available memory rows and banks can increase this dimensionality significantly. The intended base configuration contains 75 dimensions (details of the behaviour vector size calculation are given in Appendix A).

This is not an excessively high-dimensional space. For comparison, when exploring Lenia, an image-based system whose behaviour vectors can contain thousands of pixel values evolving over time, IMGEP still operates correctly.

But, even with this relatively low dimensionality, the behaviour-space bound remains extremely large: under the intended base configuration, it contains approximately 7×10^{122} possible vectors (details of the calculation are given in Appendix A). However, it is important to note that this is only an upper bound: the simulator creates strong dependencies between dimensions, so most of these combinations cannot be reached by a valid pair of programs, but the issue is that we do not know exactly how many.

The exploration algorithm is therefore never intended to cover the entire space: in Lenia, the aim is to discover a broad diversity of interesting cellular automata and trajectories. Here, we aim to reveal the intricate patterns of accesses which lead to memory interference. The role of the encoder is therefore not to describe every aspect of an execution, but to keep a compact and meaningful representation of the interference behaviours that can be discovered.

To test the modularity of the system, I implemented two alternative components. The first was a z-score encoder intended to reduce differences of scale between behaviour dimensions. The second was a QMC goal sampler based on a Sobol sequence, which distributes successive goals more evenly than independent random sampling. Figure 3.6 shows a preliminary comparison under the same exploration budget. The Sobol sampler follows the base configuration closely and does not provide a clear improvement, while the z-score encoder, alone or combined with Sobol sampling, obtains lower coverage. These results are not sufficient to draw a general conclusion about either method, but they demonstrate that alternative components can now be added and compared without changing the exploration pipeline. This confirms that the new structure is easy to modify for further experiments.

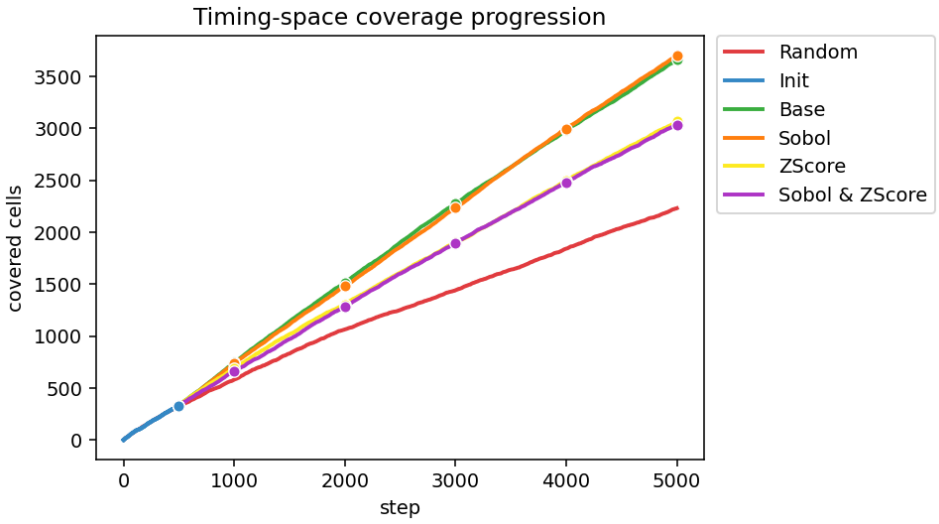


Figure 3.6: Evolution of timing-space coverage for the base IMGEP, the Sobol goal sampler, the z-score encoder, their combination, and random exploration. The initialisation phase is shared by the IMGEP configurations.

Validation against the reference implementation

Once the original features had been integrated, I needed to verify that the complete pipeline behaved correctly and remained consistent with Ludovic's implementation.

For this purpose, I added several comparison metrics to ADTool, which are presented in more detail later in this chapter, and used them to compare my runs with the results reported by Ludovic.

The comparison uses three views: the distribution of discoveries along one behaviour dimension, the coverage of two dimensions together, and the evolution of overall coverage during the experiment. In each view, IMGEP is first compared with a random baseline using the same simulator and exploration budget. I then compare the general shape of my plots with Ludovic's. The exact discoveries cannot be reproduced because randomness has an important role in the process, but both implementations should present the same general tendencies.

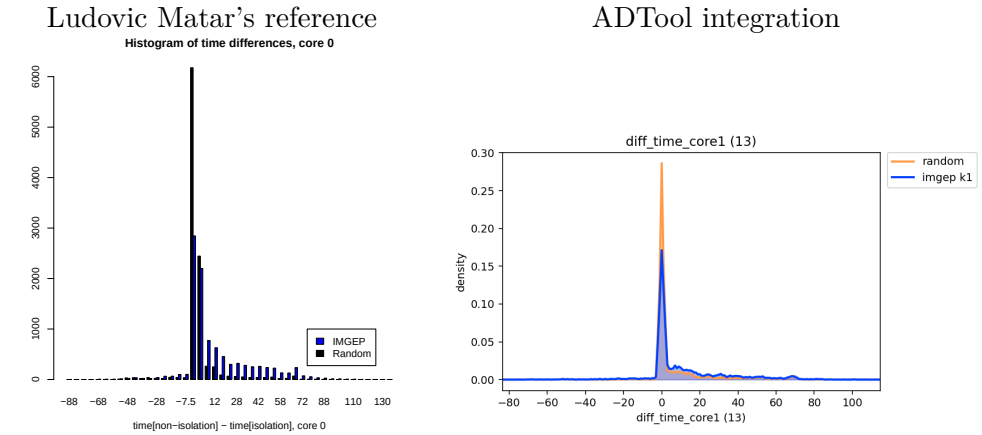
As presented in Figure 3.7, the final results are visually very similar: in both implementations, IMGEP covers the selected behaviour regions much better than random exploration. This was not the case in my first runs, where random exploration performed better, showing that the initial implementation was getting stuck. This helped me identify missing information in the behaviour encoding and in the way programs were mutated. After correcting these parts, the expected advantage of IMGEP appeared, providing evidence that the integration reproduces the intended exploration method.

Overall, my role in this first integration was not limited to transferring Ludovic's code into ADTool. I first had to understand the meaning of each metric, program operation, and exploration step before reorganising the prototype around ADTool's architecture. This restructuring separated the different responsibilities into interchangeable components, making it possible to replace them through the configuration and to experiment with new generators, mixers, mutators, goal samplers, and encoders.

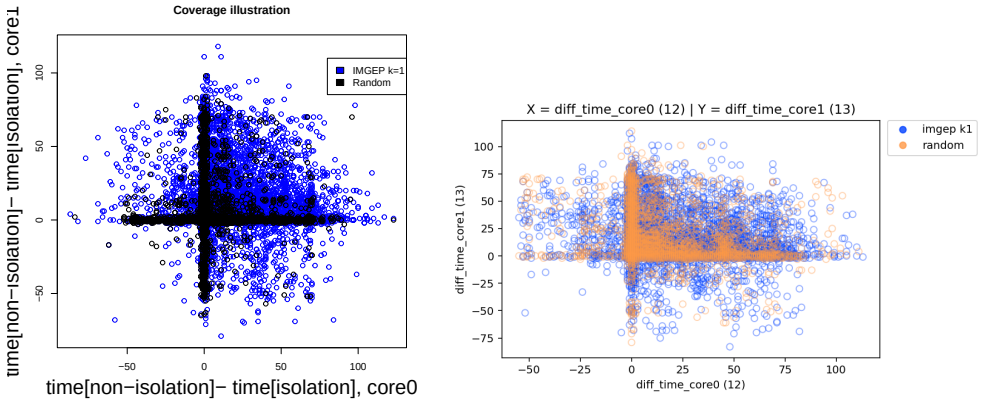
3.3 Generalising ADTool to program-based systems

While discussing this new implementation with Clément Moulin-Frier, he suggested pushing its modularity even further. The idea was to make the same structure usable by any system which takes code as input and evaluates it through a runtime environment. Therefore, I organised a higher-level architecture: a general program-based layer.

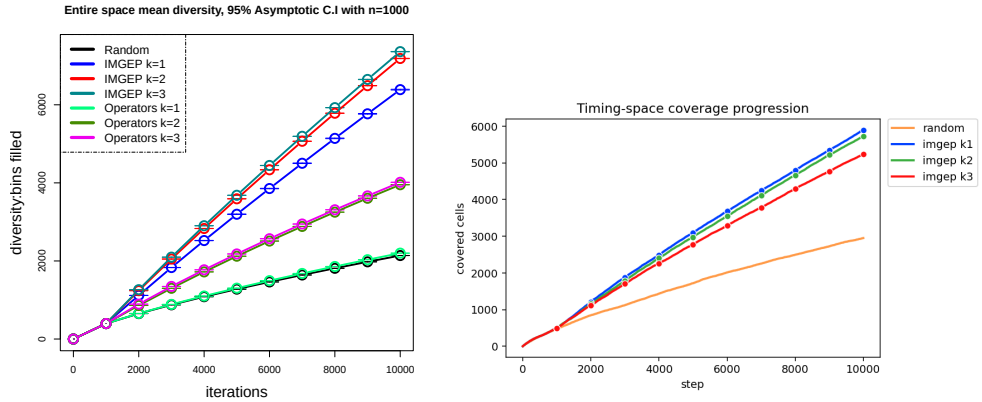
The objective of this layer is not to impose one representation of code. A program can be text, a sequence of instructions, an abstract syntax tree, or another structured object, depending on the system being studied. The question was therefore to identify which parts of the exploration could be shared to avoid reimplementing everything while retaining the freedom to define the program representation.



(a) One-dimensional coverage



(b) Two-dimensional coverage



(c) Evolution of overall coverage

Figure 3.7: Side-by-side validation of the interference implementation on the same dimensions. Each plot and its points were generated independently, using Ludovic Matar's reference module on the left and the ADTool integration on the right.

Choosing what can be reused

The explorer, the management of the parameter and behaviour spaces, and the goal-sampling mechanism all follow the same general logic. I could therefore move their base implementations into the program-based layer, where a new system can use them directly.

Other components need information which is specific to the system. A generator, mutator, or mixer has to understand the structure of the program it manipulates, while an encoder has to select outputs which are meaningful for the studied phenomenon. I consequently defined abstract classes for these components. They give each implementation the same place in the ADTool pipeline, while leaving each case study free to define the code and results it actually uses. (Table 3.1)

Level	What is provided	Examples
ADTool core	Generic experiment pipeline	Execution, history, persistence, callbacks, and analysis
Program-based layer	Reusable implementations and abstract contracts	Explorer, behaviour/parameter space handlers, goal-sampler, and interfaces for domain-specific operations
Concrete system	Domain-dependent implementations	Program type, generator, mutator, mixer, simulator, and encoder

Table 3.1: Boundary between shared program-based mechanisms and components which must remain specific to the studied system.

Selecting components through configuration

These interfaces would still be limiting if ADTool only accepted a fixed list of implementations. I therefore changed the configuration so that a component is selected from its import path rather than from a predefined name or a boolean option. As long as the selected class follows the expected abstract interface, ADTool can load each component with its own specific configuration. This makes it possible to add or replace a generator, mutator, mixer, encoder, goal sampler, or simulator without modifying the exploration pipeline while maintaining the original reusability and simplicity of the architecture.

The interference study remains the original case for which this work was designed, but it now also serves as a complete example of how to implement a new program-based system. It shows how to define the program representation, implement the required components, connect a simulator, and configure the shared exploration structure.

3.4 Reengineering ADTool

Once the interference study was integrated, several problems from ADTool itself became much more visible. Some features were missing, others were no longer working, and parts of the library made experiments unnecessarily slow or hard to reproduce. I therefore suggested reengineering the library around its actual scientific use.

This section remains voluntarily brief. These changes took time, but explaining every internal modification would be quite heavy to read and would turn this report into technical documentation. I instead present what was causing a problem, what the library was expected to do, and how I changed it. The complete usage and extension details are available in the documentation I wrote alongside the implementation [15].

Library structure and reliability

Before this work, importing ADTool also required importing its examples. This meant installing the dependencies of every available system, even when only the main library was needed. The package definition was also based on an outdated structure, while core modules, example-specific code, and unused components were mixed throughout the repository.

Therefore, I replaced the deprecated `setup.py` file with a `pyproject.toml`, which allows ADTool to be installed as a standalone library, with example dependencies remaining optional. I then cleaned the repository to create a clear separation between the reusable code and the examples. Following the same idea as for program-based systems, I also applied a more regular modular structure to the rest of the library. Components are now loaded and configured in a consistent way, which makes the codebase cleaner and easier to extend.

There were previously no automated tests to check that these modifications did not break the existing examples. I therefore added small smoke tests which run each available system for a few steps and verify that it still produces valid discoveries. These tests turned out to be very useful regression checks, allowing me to catch issues introduced by my own edits.

History and checkpoints

The history was the most important internal issue. IMGEP constantly uses past discoveries to select behaviours and recover the corresponding parameters. The previous implementation tried to reduce memory usage through a database and repeated serialisation but, in practice, this feature was not working: it performed many costly operations before falling back to the data already kept in memory. Additionally, as the

number of discoveries increased, this implementation also created considerable slowdowns. Finally, the checkpoint feature, which should have saved experiment state and allowed experiments to resume, was also missing.

When rewriting the history for the first time, I did not know about the low memory and checkpoint requirements. The initial implementation fixed the immediate slowdown, but kept everything in RAM and was not saving all the internal states required to properly continue an experiment. After discussing this with Clément Romac, I reworked it around checkpoints. After a configurable number of steps, ADTool now saves the new batch of discoveries together with the state of every experiment component. A run can therefore be resumed from that point, or used as the common starting point of several new branches without modifying the original one.

This is particularly useful for scientists because it makes it possible to analyse an exploration while it is running, then to change its direction without losing the work already done. They can return to an earlier checkpoint, modify the goal sampler or mutator, and launch a new continuation. The original run and every new branch remain stored separately, making the different configurations easy to navigate and compare as illustrated in Figure 3.8.

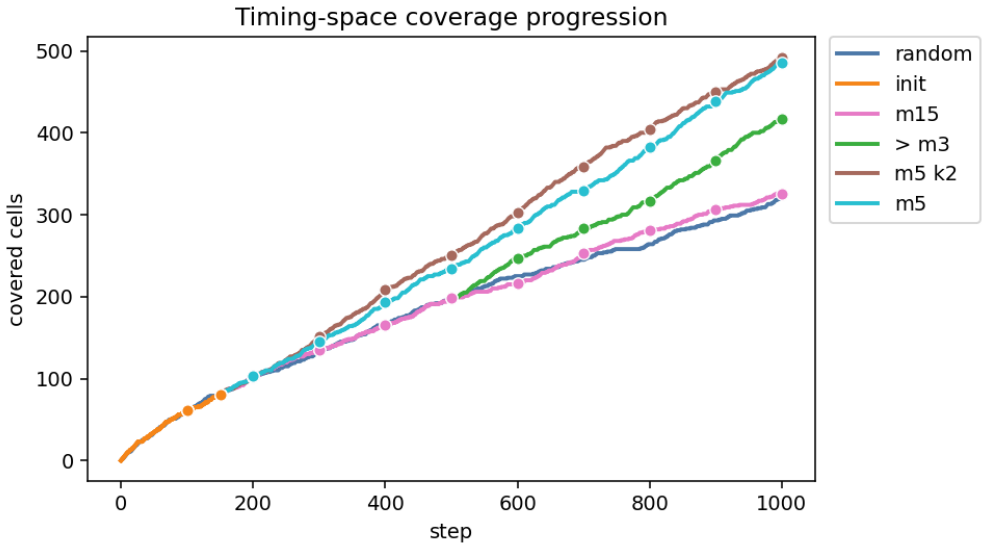


Figure 3.8: Example of several explorations branching. A point is a checkpoint and each color is a different configuration. Progression of each configuration's space-coverage can then be compared from their respective checkpoint paths.

The rewritten history balances memory use and retrieval time, as summarised in Figure 3.9. New discoveries accumulate in RAM until the next save, which adds a checkpoint on top of the existing stack. This checkpoint owns one new history chunk containing the current batch. The stack of checkpoints and chunks therefore

progressively forms the complete history. During retrieval, the explorer compares the recent data kept in RAM with the data retrieved sequentially from every older chunk to find the best matches across all these sources. This avoids keeping the complete history in RAM at the cost of reading multiple files, which can slow down the process.

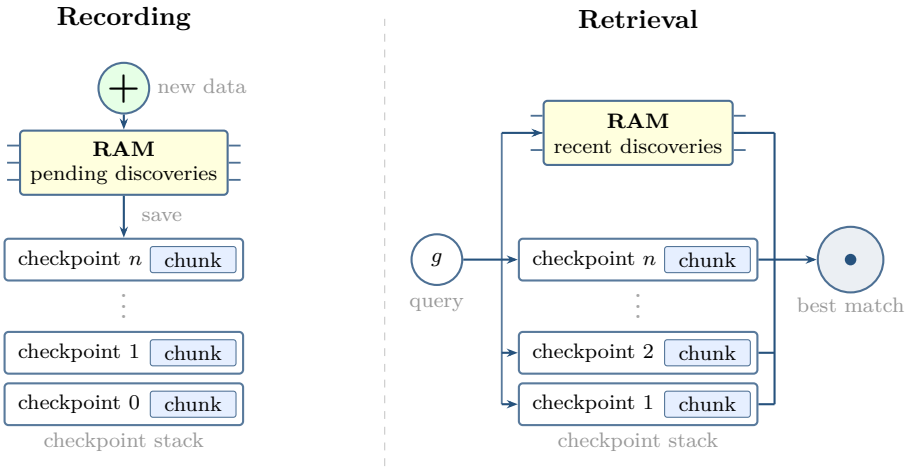


Figure 3.9: History recording and retrieval. Each checkpoint contains one history chunk. A save adds a checkpoint and its chunk to the top of the stack. A query searches RAM and every stored chunk before returning the best matches.

Visualisation and analysis

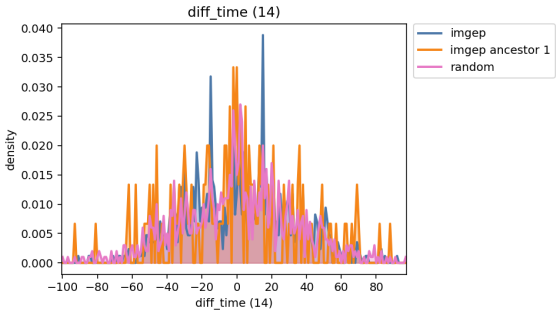
According to Clément Romac, an earlier version of ADTool used a server to interact with running experiments. However, I never had access to this version and only found a few screenshots in the old documentation. A local visualisation server was still present (Figure 2.8), but it only offered a limited view of the saved discoveries and was not sufficient to study the result of a large exploration. I therefore created a new interface from the beginning.

The server now organises discoveries in an interactive two-dimensional map. It can follow a running experiment, pause and resume it, inspect its checkpoint branches, highlight behaviours, and select or export interesting discoveries. It can also define goal zones to guide a compatible sampler toward regions chosen by the scientist. These features remain generic enough for systems which produce images, videos, or only numerical vectors. The main parts of this interface are illustrated in Appendix B.

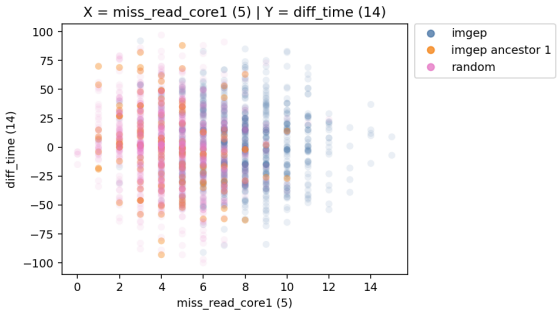
But visualisation alone was not sufficient: ADTool previously had neither a stable plotting tool nor a comparison tool. It was consequently difficult to know whether an IMGEP run was exploring well, performing worse than random, or discovering a meaningful variety of behaviours. I added a separate analysis module for this purpose. This required saving the numerical data needed after a run, then linking the module

to the server so that analyses can be launched and inspected without writing a new script each time.

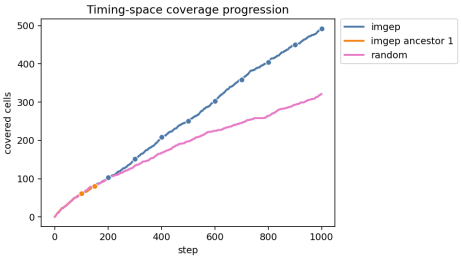
I first added broad analyses which can be used with almost any system: one-dimensional and two-dimensional comparisons, and the evolution of behaviour-space coverage, all comparing the selected results with a random baseline. These are the metrics I used earlier to validate the interference integration. However, each case study can also require more specific information, so I once again made the structure modular. New analysis modules can be added without changing the general analysis runner or the server. As a first example, I implemented a module for the interference study which tracks DDR accesses and displays them as heatmaps. (Figure 3.10)



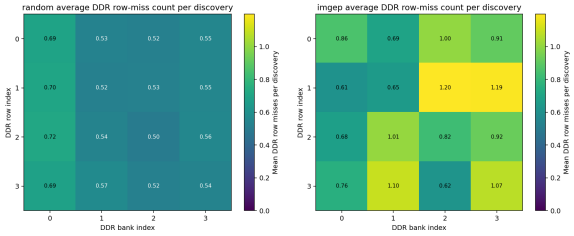
(a) One-dimensional comparison



(b) Two-dimensional comparison



(c) Behaviour-space coverage progression



(d) Custom interference metric: DDR mutual-miss heatmaps

Figure 3.10: Analysis metrics and views available in ADTool

Finally, I documented the installation, configuration, checkpoint system, analysis modules, visual interface, and the process for adding a new system. This documentation was important because the objective of the reengineering was not only to make ADTool work again, but also to make it understandable and usable by other scientists [15].

At the time of writing, this work is still ongoing. While I was working, Ludovic Matar continued to update his interference study, and I am now helping him migrate

these changes to the reworked ADTool structure. We also continue to discuss possible directions for improving IMGEP performance.

3.5 Resulting contribution to ADTool

The internship subject proposed adapting the existing interference prototype to ADTool: the simulator and exploration components had to be integrated into the library, then evaluated through diversity measures [2]. I followed this direction, but the final contribution became larger because the integration also revealed what was missing for ADTool to be used comfortably on this new system.

The first result is a complete interference-identification experiment inside ADTool. It takes two low-level programs as parameters, executes them in the simulator, encodes their interference behaviour, and uses IMGEP to discover new patterns. It can be configured, compared with a random baseline, analysed, and inspected through the same workflow as the other ADTool systems. The validation shows that, for this simulator and configuration, the integrated IMGEP produces coverage results similar to Ludovic Matar's reference implementation.

I also implemented the more general program-based system behind this example. Instead of treating the interference programs as a one-time exception, ADTool now has a reusable structure for systems which use code as their input. A new system can define its own program format, simulator, generator, mixer, mutator, and behaviour encoder, while reusing the generic exploration pipeline. The interference study is therefore both a concrete use case and a complete example of how another program-based experiment can be added to the library.

Finally, the reengineering work translates the original idea of ADTool into a more usable tool for scientists. Its core is separated from its examples and can be imported without their dependencies; experiments can be saved, resumed, and branched through checkpoints; results can be explored and analysed through the new interface; and smoke tests help ensure that later modifications do not break the existing systems. The complete documentation makes these mechanisms easier to understand and reuse.

Overall, I did not only add one simulator wrapper. I implemented a new type of system in ADTool, used it to make the interference prototype accessible through the normal exploration tools, and reworked the library so that this use can be continued, analysed, and extended. The internship and the migration work are still in progress at the time of writing. Chapter 4 therefore presents future work: further ideas for extending the library, possible fixes for bugs that may appear as it is used on new systems, and directions that I did not have time to implement during the internship.

4. Discussion

4.1 Contribution to the AIxIA project

The work provides AIxIA with a practical environment in which the interference experiment can be continued, adapted, and inspected. Instead of remaining tied to one prototype, the experiment now benefits from the normal ADTool workflow for configuration, history, comparison, analysis, and visualisation. This makes it easier for researchers to keep track of an exploration and to discuss the results using the same stored discoveries and views.

It also creates a useful basis for the next scientific experiments. Each module can now be changed independently, so a new behaviour representation, program operator, simulator, or goal sampler can be tried without having to rewrite the system core. The program-based layer further means that this approach can be reused when another system takes code or structured instructions as input. The interference system is therefore both the application which motivated the work and a starting point for future code-based experiments in ADTool.

4.2 Collaboration and team interactions

This work was shaped by regular discussions with several people, rather than by a strictly isolated implementation process. Ludovic Matar was the main point of contact for the interference study and IMGEP. Our discussions helped me understand the reference implementation, interpret and create coverage metrics, and identify potential issues regarding the expected behaviour. He also helped me keep the implementation connected to the scientific objective: discovering different interference behaviours.

Although I only had occasional exchanges with Clément Moulin-Frier, he encouraged me to generalise the first interference-specific implementation into a program-based system. Clément Romac helped me understand the previous organisation of ADTool and discussed the intended role of features such as history, configuration, or the web interface.

At first, I was free to edit the library directly without a formal development process. I asked to organise the work more like a regular project instead, by creating issues for the work to be done and submitting the changes through pull requests to be reviewed by Clément Romac. This gave a clearer record of the decisions taken throughout the internship and made it easier to discuss the implementation ideas with the team.

During the internship, I also had the chance to collaborate with a team of researchers from IRCAM (Institut de Recherche et Coordination Acoustique/Musique). Clément Moulin-Frier first presented the tool to the team and Antonin Longeot, supervised by Nicolas Obin, then used ADTool to study an abstract space generated by a synthesiser. This was an especially interesting example because it showed that ADTool could already be used by another group for a completely different system. Helping Antonin understand the library and set up this project made ADTool's objective as a reusable tool much more concrete. Seeing my contribution become useful in this new context was also personally very rewarding regarding the work completed during the internship.

4.3 Critical reflection and limitations

The main remaining limitation of the current ADTool workflow concerns the evaluation of individual components. The analysis metrics I added give a broad view of a complete exploration: they can compare coverage, inspect selected dimensions, and show the progression of a run. However, they do not directly answer more specific questions, such as whether one mutator produces a broad range of candidates, or whether a goal sampler really improves behavioural diversity. Defining a metric for each component is not straightforward because two systems can organise their exploration in completely different ways. For example, one system may omit the evolutionary mutation step and use reinforcement learning instead, while another may generate new parameter candidates with a decoder rather than retrieving the nearest neighbour. A metric that is meaningful for one implementation may therefore not apply to another one. I unfortunately thought of this idea late in the internship, during a discussion with the IRCAM team, and did not have time to implement it.

A second limitation is the absence of systematic execution-time benchmarks for the complete workflow. The history is the main potential bottleneck because goal sampling and nearest-neighbour retrieval repeatedly consult a growing number of discoveries, sometimes requiring checkpoint chunks to be read from disk. The new implementation reduces this cost, but I did not establish how its execution time and memory use evolve with the history size, cache size, checkpoint frequency, and behaviour-vector dimension. Such benchmarks would make it possible to locate the remaining slowdowns and choose appropriate defaults for larger experiments.

The current smoke tests are sufficient to check that every example can run and produce a valid discovery, but they remain limited by the cost of the systems themselves. Some simulations are too expensive to run for many iterations in a general test suite. More rigorous tests should therefore target the methods directly with small controlled inputs. For the history, they could verify record ordering, cache and look-

back limits, exact nearest-neighbour retrieval, and checkpoint continuation without executing a complete system. This would give stronger guarantees while avoiding the computational constraints of the most expensive examples.

There is also room for a higher-level reflection on the configuration itself. The current workflow allows the researcher to choose the mutator settings, the observations retained in the behaviour vector, and the way goals are sampled, but it does not help decide which of these choices is the most appropriate for a given study. Finding ways to propose values for these settings, choose relevant observations, or adapt the module structure more autonomously would be an interesting research direction, not only for ADTool but for the IMGEP itself. However, this would require a separate reflection on the scientific objective of each system, rather than only another change to ADTool. These are minor limitations which do not affect the results of the base workflow, but they provide useful directions for later work.

4.4 Future work and ADTool outlook

With these possible extensions in mind, the next step is to introduce the tool to Ludovic Matar and accompany him in moving to ADTool. We are also still discussing possible improvements to the interference case study. For example, one direction that I could explore is a reinforcement-learning-based method. This idea, together with other possible changes to the study, is still being discussed at the time of writing. It is therefore a potential next task, not a result that can already be claimed.

The ADTool repository also contains open issues proposing additional features [7]. Some of them concern code optimisation and structure rework. Others would add metrics to check whether a mutator produces sufficiently diverse programs, whether a goal-sampling strategy actually encourages behavioural diversity, and whether a run discovers pertinent behaviours rather than only many noisy points. The visual interface could also be improved with meaningful axis names, clearer colours for goals and their progression, the ability to select discoveries as goals, and controls to pause the projection refresh or execute one or several IMGEP steps. These are good ideas for later versions of the library. However, Clément Romac advised me not to attempt all of them at the end of the internship: they are not mandatory for the tool to be used, and my remaining time can be spent on a more interesting research direction for the interference study.

Overall, the outlook for ADTool is to continue becoming a practical reusable tool for researchers. The current interference study and the collaboration with IRCAM already provide two different use cases, while the remaining ideas give clear ways to make future experiments easier to evaluate and control.

5. Professional Development and Reflections

5.1 Scientific activities and professional development

This internship helped me understand how a research team works on a daily basis. Beyond the technical work, the weekly AIxIA meetings helped me understand that a research project is not only about implementing one solution: it also requires keeping track of assumptions, explaining partial results, and changing direction when a result raises a new question.

I also attended the weekly FLOWERS team meetings, several talks on curiosity and evolutionary AI, and a reading club led by Julien Rosenberger in the laboratory. The reading club was particularly useful for me. It taught me how to read papers more quickly without trying to understand every detail, discuss their limits with other researchers, and take more structured and persistent daily notes. The discussions around papers, AI, and ethics also showed me how useful it is to have people from different specialities look at the same question.

The discussions, meetings, and presentations also gave me a much broader view of the research carried out in the team. I learned about cognitive science, including curiosity, developmental progress, and learning progress, as well as artificial life through systems such as Lenia, evolutionary AI, multi-agent environments, world models, and LLMs. These experiences familiarised me with fields unrelated to my own subject and showed me that an idea from a completely different topic can still be useful. For example, when Marko Cvjetko showed me some of his writing on Lenia, it made me think of new metrics that I could propose for my own work.

Beyond the formal meetings, I had many informal discussions with other FLOWERS members, PhD students, and interns. I tried to give ideas when I could and often volunteered when somebody needed participants for a human-interaction experiment. Paul, Timothé, Julien, Jérémy, Marko, Loris, Miguel, Adèle, or Léandre are some of the people who made these discussions natural, whether about their thesis, more abstract scientific questions, or daily subjects unrelated to work. This welcoming environment made the internship much more enjoyable than working only on my own subject. It also reinforced my interest in a research career. Before this internship, I

was still unsure whether I wanted to continue in academia or move directly to a more conventional engineering job. Working with the FLOWERS team made me want to continue in research.

5.2 Social responsibility, ethics and sustainable development

The project can have positive social value because it addresses timing problems in critical embedded systems. Better ways to identify interference could help engineers analyse systems used in domains where timing errors may have serious consequences, such as aeronautics or robotics [2], [5].

More broadly, machine-learning and deep-learning systems increasingly aim to automate tasks that were previously carried out by humans. The objective here is different. IMGEP explores the system and finds diverse, interesting behaviours that can later be analysed by experts. It does not try to interpret these behaviours, identify their causes, or choose a correction. These steps still depend on engineers and hardware specialists. ADTool should therefore be seen as a tool to help human analysis, not to replace it.

This method can also be seen as a more sustainable direction than the LLM-based approach explored in parallel by the Toulouse team, which trains a model to study execution traces. IMGEP does not need to train a large model or collect a large training dataset beforehand. Instead, it runs simulations and uses the behaviours it has already observed to select the next experiments. This makes the curiosity-driven approach interesting when a full training phase is not necessary. Making ADTool easier to reuse can also avoid duplicated engineering work by allowing other researchers to build on the same exploration framework.

Unfortunately, I do not have metrics for these social, ethical, and sustainable impacts, because they were not the main objective of the project. However, considering these aspects is still useful: it shows that the choice of an exploration method can have consequences beyond its direct scientific results.

5.3 Use of coding agents and reflections on AI

Coding agents were part of my working process during this internship. Before using one, I made sure to ask whether the project contained private information that should not be shared with an external tool. The team explained that ADTool and AIxIA are open projects, and that an AI assistant could actually be quite useful. This was important because I remained responsible for confidentiality and for the information

shared with the tool.

At first, I used GitHub Copilot, not because I thought it was the best agent, but because it was free for students when I started using it. It quickly became clear that it was not suited to all my needs, so I started looking at other models. I first discussed this with the FLOWERS team, where almost everyone was using an Anthropic model. I then compared the benchmarks available online, read documentation for the different agents, and looked at reviews and discussions on social media. BenchLM was one of the benchmark websites I used for this comparison [20]. This did not give me a complete or scientific comparison, but it helped me understand which tools were being used and which ones could fit my own workflow.

However, the choice was no longer obvious when I was looking for an agent. The Claude subscription options available to me were more expensive than I wanted and their usage limits appeared restrictive for my planned use. At the same time, Gemini from Google, ChatGPT Codex from OpenAI, Kimi from Moonshot, Qwen from Alibaba, and GLM from Zhipu were becoming serious candidates. Their online benchmarks, reviews, and discussions did not always agree, and the tools were changing quickly. I therefore could not treat one benchmark or one review as a direct ranking, but had to decide which compromise made the most sense for my own use.

Kimi was my first choice because it was open-weight and its offer fitted my budget. Its online benchmark results and the reviews I found made it appear to be a serious candidate for coding tasks. Unfortunately, I could not subscribe to it or test it at that time.

In the end, I chose ChatGPT Codex because its offer fitted my budget and usage, and because I could use it both on the web for daily personal use and in my development environment. Its online benchmark results, documentation, and reviews also made it appear well suited to the kinds of tasks I needed: reading an existing codebase, preparing a small modification, proposing an implementation plan, and helping with straightforward interface work.

My usual method was first to understand the code and the change I wanted to make, then ask for a small and precise task, review every modification, and test the result. This was necessary because the agent often made mistakes or missed part of the intended behaviour, even when its answer initially looked plausible. I still had to correct these changes myself. I therefore needed to understand the subject well enough to notice when an implementation was incomplete, incorrect, or simply not what I had asked for. This was especially important for the history rework and the generic parts of ADTool, where a plausible-looking change could easily break another system. In the end, I spent less time writing repetitive code, but more time planning the change, reviewing it, and checking whether it really answered the scientific or software-engineering question. The agent could help write and organise, but I remained

responsible for the idea, the verification, and the final decision.

I also spent time discussing the broader consequences of these systems with members of the team, both informally and during the reading club. These discussions made me think that agents are tools, and that, like any tool, they can be used in better or worse ways. Learning how to use them correctly is therefore important. This is still difficult because the tools and the ways to use them are evolving quickly, but introducing them and teaching their limitations seems more useful than pretending that they do not exist.

This question is particularly important in education. Students first need to actually struggle with a task and learn how to solve it before they can automate part of it. This does not only concern code-generation agents, but more general AI assistants as well. Through the discussions on cognitive science in the lab, I understood that these tools could have very positive effects, but also very negative ones if they prevent people from developing their own understanding. They should therefore be introduced carefully and studied further.

In research, I read blogs and posts about these tools and discussed their use with other researchers. My conclusion is the same: the most important question is not only whether we should use them, but how we use them. I think that these systems can be of great help to humankind, but they are still recent and need to be approached with care. The current rush for better performance worries me a little. In my opinion, a slower development that gives more importance to ethics, education, and safety would be a better direction.

6. Conclusion

This internship took place in the context of AIxIA, a project that studies how AI can help analyse temporal interference in multicore embedded systems. Its initial objective was to integrate a curiosity-driven exploration method into ADTool in order to generate and inspect diverse interference behaviours. This required connecting low-level programs, a CPU simulator, a behaviour representation, and an IMGEP exploration loop inside the same reusable workflow.

I first implemented the memory-interference experiment in ADTool, then generalised it into a program-based system able to explore systems whose parameters are code or structured instructions. This made the interference study the first use case of a more reusable abstraction instead of a one-off implementation. During this work, I also found that ADTool needed a wider rework before it could be used comfortably by other researchers. I therefore repaired imports and history handling, separated generic and example-specific code, added checkpoints, analysis metrics, documentation, smoke tests, and a clearer visual interface. This made ADTool easier to reuse, analyse, view, and extend for future studies.

The interference integration was examined through the new analysis metrics and through feedback from Ludovic Matar, while the library changes were confirmed through smoke tests and reviews by Clément Romac. These positive results support the conclusion that the integration can be analysed through the normal ADTool workflow and that the rework retained the expected behaviours while offering new features.

Beyond the technical work, this internship was also a very interesting research experience. I had to learn how to approach a subject that was initially far from my own knowledge, read and use papers efficiently, and discuss my work with researchers from different fields. The discussions with the FLOWERS team and the Toulouse partners were especially useful because they showed me how a question can be approached from different perspectives and how an idea from an apparently unrelated field can lead to a useful new direction. This experience clearly reinforced my desire to continue in research.

The interference experiment now gives AIxIA a useful basis for future studies, while the program-based layer may make ADTool useful for other complex systems.

Bibliography

[1] C. Courtaud, "Caractérisation de la sensibilité aux interférences mémoire dans les systèmes temps-réels embarqués sur des plateformes multi-coeurs,"

[2] L. Matar and C. Moulin-Frier, *Curiosity-driven automated discovery of interference patterns in multi-core architectures*, Master-level internship subject, 2026.

[3] "Flowers AI and CogSci lab, artificial intelligence, cognitive sciences, education and scientific discovery. "[Online]. Available: https://flowers.inria.fr/

[4] "IRT saint exupéry," Methods & tools for the development of complex systems. [Online]. Available: https://www.irt-saintexupery.com/fr/methodes-outils-des-systemes-complexes/

[5] E. JENN, "L'intelligence artificielle au service de l'IngénieRie des SysTèmes aéRonautIques et sPatiaux (AIRSTRIP)," Internal document, 2023.

[6] L. MATAR, *Source interference identification simulator*. [Online]. Available: https://github.com/Ludovicccccc/SourceInterferenceIdentificationSimulator

[7] FLOWERS Team, *ADTool: Assisted and automated discovery for complex systems*. [Online]. Available: https://github.com/flowersteam/adtool

[8] G. BRAU, "State of the art in interference identification techniques," IRT Saint Exupéry, State of the Art, Aug. 22, 2024.

[9] L. MATAR, "Application of curiosity-driven exploration algorithms for hardware interference identification," Unpublished.

[10] T. Lugo, S. Lozano, J. Fernández, and J. Carretero, "A survey of techniques for reducing interference in real-time applications on multicore platforms," *IEEE Access*, vol. 10, pp. 21 853–21 882, 2022, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3151891 [Online]. Available: https://ieeexplore.ieee.org/document/9714355

[11] A. Modirshanechi, K. Kondrakiewicz, W. Gerstner, and S. Haesler, "Curiosity-driven exploration: Foundations in neuroscience and computational modeling," *Trends in Neurosciences*, vol. 46, no. 12, pp. 1054–1066, Dec. 1, 2023, ISSN: 0166-2236. DOI: 10.1016/j.tins.2023.10.002 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0166223623002400

[12] S. Forestier, R. Portelas, Y. Mollard, and P.-Y. Oudeyer, "Intrinsically motivated goal exploration processes with automatic curriculum learning," *Journal of Machine Learning Research*, vol. 23, no. 152, pp. 1–41, 2022, ISSN: 1533-7928. [Online]. Available: http://jmlr.org/papers/v23/21-0808.html

[13] E. N. Lorenz, "Deterministic nonperiodic flow," *Journal of the Atmospheric Sciences*, vol. 20, Mar. 1, 1963, ISSN: 0022-4928, 1520-0469. DOI: 10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2 [Online]. Available: https://journals.ametsoc.org/view/journals/atsc/20/2/1520-0469_1963_020_0130_dnf_2_0_co_2.xml

[14] M. Etcheverry, "Curiosity-driven AI for science : Automated discovery of self-organized structures," Ph.D. dissertation, Université de Bordeaux, Nov. 16, 2023. DOI: 10.70675/5041450bz7b8az4fcez966azf89b0a4e7e48 [Online]. Available: https://theses.hal.science/tel-04504878

[15] A. MACDONALD. "ADTool technical documentation. "[Online]. Available: https://github.com/flowersteam/adtool/tree/main/docs

[16] M. Etcheverry, C. Moulin-Frier, P.-Y. Oudeyer, and M. Levin, "AI-driven automated discovery tools reveal diverse behavioral competencies of biological networks," *eLife*, vol. 13, Aug. 20, 2024. DOI: 10.7554/eLife.92683.3 [Online]. Available: https://elifesciences.org/reviewed-preprints/92683

[17] FLOWERS Team, *ADTool: Historical implementation*, Jan. 30, 2025. [Online]. Available: https://github.com/flowersteam/adtool/tree/d9e61f85605b98320847a888d0085dc01edd3a40

[18] N. Binkert et al., "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, 2011, ISSN: 0163-5964. DOI: 10.1145/2024716.2024718 [Online]. Available: https://dl.acm.org/doi/10.1145/2024716.2024718

[19] I. M Sobol', "On the distribution of points in a cube and the approximate evaluation of integrals," *USSR Computational Mathematics and Mathematical Physics*, vol. 7, no. 4, pp. 86–112, Jan. 1, 1967, ISSN: 0041-5553. DOI: 10.1016/0041-5553(67)90144-9 [Online]. Available: https://www.sciencedirect.com/science/article/pii/0041555367901449

[20] BenchLM. "LLM leaderboard and AI model benchmarks. "[Online]. Available: https://benchlm.ai/

Appendix A. Space size calculations

Symbol	Meaning	Base config value
D	Number of dimensions in the encoded vector	75
N_{timing}	Number of retained timing dimensions	3
N_{DDR}	Number of retained DDR matrices	4
N_{row}	Number of reachable simulated DDR rows	4
N_{bank}	Number of simulated DDR banks	4
N_{L2}	Number of retained L2 counters	8
$\lvert\mathcal{B}\rvert_{\text{box}}$	Rectangular upper bound of the behaviour space	7.06×10^{122}
N_{instr}	Number of instructions in one generated program	25
T_{solo}	Maximum completion time of an isolated execution	128
T_{mutual}	Maximum completion time of a mutual execution	256
$\lvert\mathcal{P}\rvert$	Number of possible initial program pairs	1.28×10^{143}
N_{cycle}	Number of available execution cycles	128
N_{address}	Number of addresses available to one core	32

A.1 Size of the vector retained by the encoder

$$D = N_{\text{timing}} + N_{\text{DDR}}\,(N_{\text{row}} \times N_{\text{bank}}) + N_{\text{L2}}. \tag{A.1}$$

For the base configuration:

$$D_{\text{base}} = 3 + 4\,(4 \times 4) + 8 = 75. \tag{A.2}$$

A.2 Upper bound on the number of encoded behaviour vectors

$$\begin{aligned} |\mathcal{B}|_{\text{box}} = {} & \underbrace{(2N_{\text{instr}}+1)^{\frac{N_{\text{DDR}}N_{\text{row}}N_{\text{bank}}}{2}}}_{\text{DDR differences } [-N_{\text{instr}},N_{\text{instr}}]} \underbrace{(N_{\text{instr}}+1)^{\frac{N_{\text{DDR}}N_{\text{row}}N_{\text{bank}}}{2}}}_{\text{DDR mutual-only } [0,N_{\text{instr}}]} \\ & \times \underbrace{(3N_{\text{instr}}+1)^{N_{\text{L2}}}}_{\text{L2 differences } [-N_{\text{instr}},2N_{\text{instr}}]} \underbrace{(T_{\text{solo}}+T_{\text{mutual}}+1)^2}_{\text{timing differences } [-T_{\text{solo}},T_{\text{mutual}}]} \\ & \times \underbrace{(2T_{\text{mutual}}+1)}_{\text{mutual timing } [-T_{\text{mutual}},T_{\text{mutual}}]}. \end{aligned} \tag{A.3}$$

For the base configuration:

$$\begin{aligned} |\mathcal{B}_{\text{base}}|_{\text{box}} &= (2\times 25+1)^{\frac{4\times4\times4}{2}}\,(25+1)^{\frac{4\times4\times4}{2}} \\ &\quad \times (3\times 25+1)^8\,(128+256+1)^2\,(2\times 256+1) \\ &\approx 7.06 \times 10^{122}. \end{aligned} \tag{A.4}$$

A.3 Number of possible initial program pairs

$$|\mathcal{P}| = \left[\binom{N_{\text{cycle}}}{N_{\text{instr}}} (2N_{\text{address}})^{N_{\text{instr}}} \right]^2 . \tag{A.5}$$

For the base configuration:

$$|\mathcal{P}_{\text{base}}| = \left[\binom{128}{25} (2 \times 32)^{25} \right]^2 \approx 1.28 \times 10^{143}. \tag{A.6}$$

Appendix B. Visualisation and analysis interface

The interface combines several tools intended to help scientists navigate, interpret, and reorient an exploration.

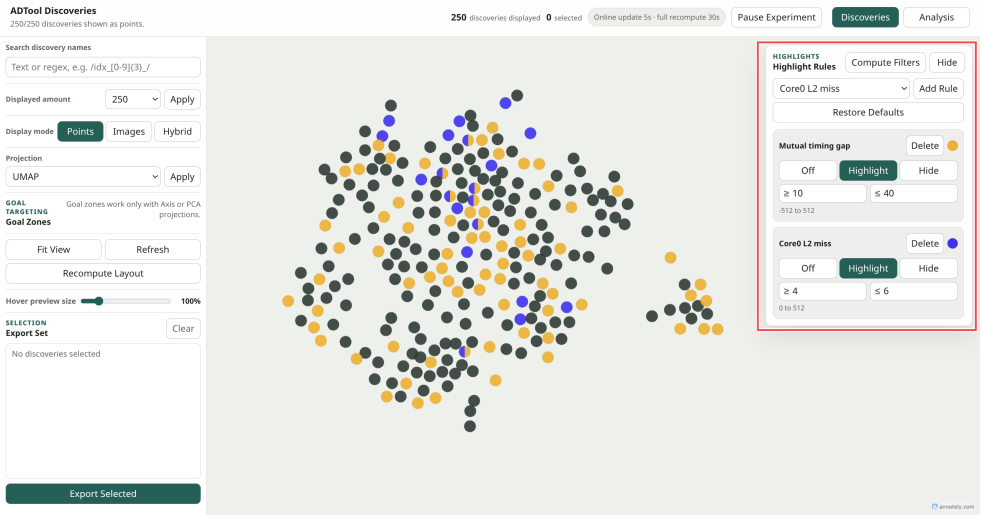


Figure B.1: **Filters and highlights.** The user can define rules from the values of the behaviour vector, then colour or hide the corresponding discoveries. This makes possible clusters and tendencies easier to identify in the projection.

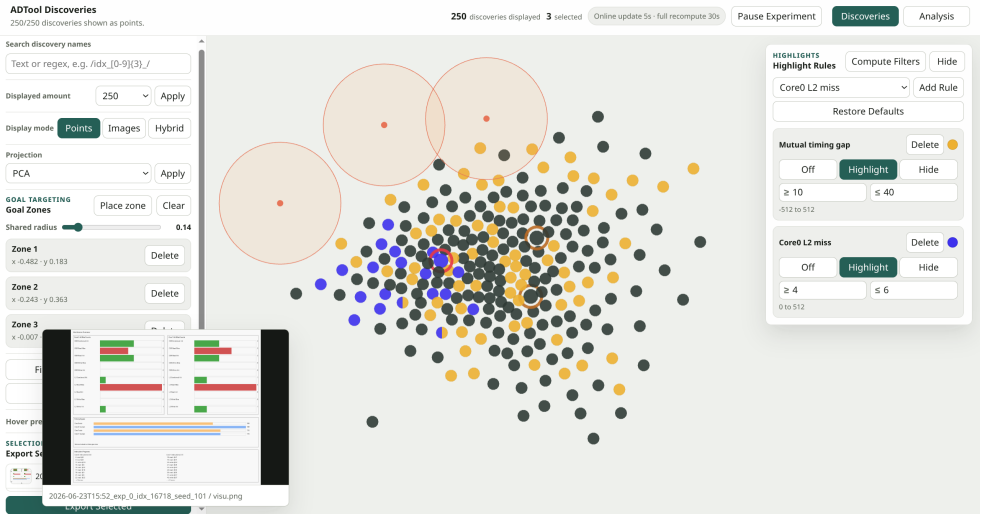


Figure B.2: **Goal targeting.** The user can draw zones directly in the projected space. A compatible goal sampler then orients the exploration towards these regions while continuing to explore outside them.

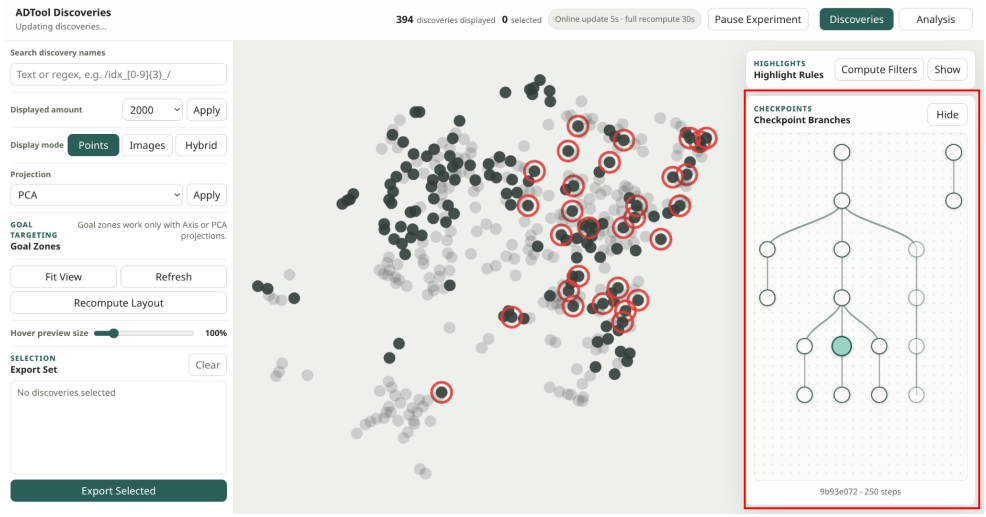


Figure B.3: **Checkpoint navigation.** The checkpoint tree can highlight complete exploration paths and separate their branches. It helps to see the directions taken by each continuation and how their discoveries evolve over time.

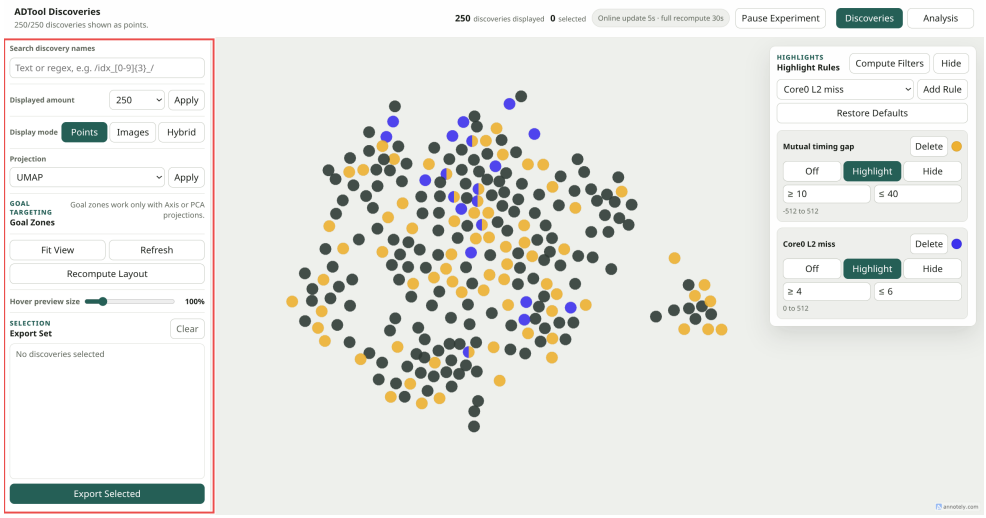


Figure B.4: **Navigation tools.** The toolbar groups the controls used to filter discoveries, change how many are displayed, adapt their visual representation, select another projection, and navigate through the map.

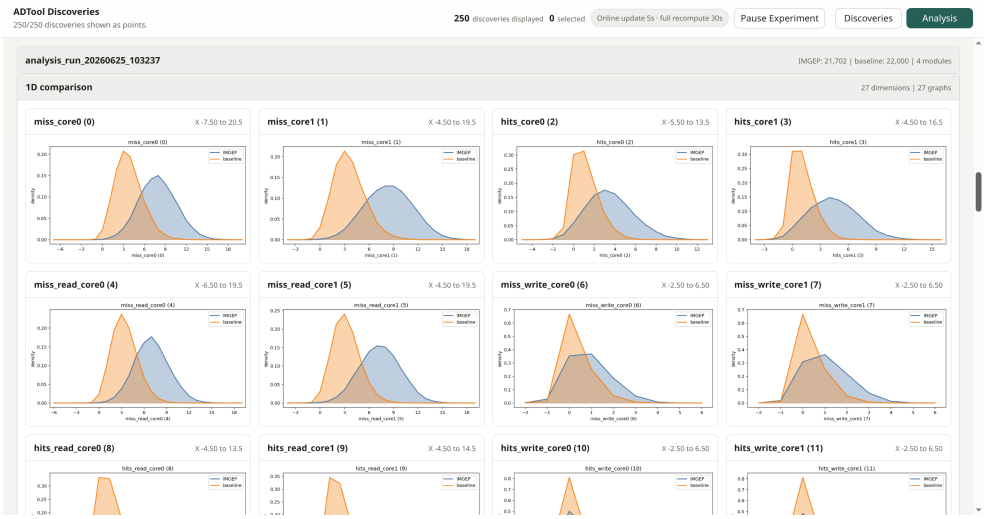


Figure B.5: **Analysis page.** Scientists can launch configured analysis modules from the interface and inspect their plots interactively. This keeps the analysis workflow in the same place as the explored discoveries.